Implementationsprompt för Replit 3.0: Objektdata & Metadata-system för Kinab

Skapad: 2026-01-13
För: Replit 3.0 Agent
Projekt: Kinab Ordersystem
Fokus: Implementation av objektdata och metadata enligt Mats vision

---

Del 1: Kontext och Mål

1.1 Sammanfattning av Mats Vision

Mats har tydligt definierat en fundamental arkitektur för hur objektdata och metadata ska separeras och organiseras i Kinabs nya ordersystem.

Kärnprinciper

Princip 1: Håll Objektet Enkelt

> Objektet är bara en minimalistisk container - ett skal med endast fyra attribut. All komplexitet ska ligga i metadatan som kopplas till objektet.

Princip 2: Blanda Inte Äpplen och Päron

> Det nuvarande Modus-systemet blandar objektdata (ID, namn, typ, förälder) med metadata (beskrivning, antal, kund, artikel) i samma struktur. Detta gör systemet tungt, svårt att underhålla och svårt att skala.

Princip 3: Metadata är Separata Entiteter

> Metadata ska inte vara kolumner i objekttabellen, utan separata poster som pekar på objektet. Detta ger flexibilitet att lägga till nya metadatatyper utan att ändra databasstrukturen.

Princip 4: Intelligens för Automatisk Kategorisering

> Systemet ska ha intelligens för att automatiskt kategorisera och koppla metadata till rätt objekt baserat på kontext och regler.

1.2 Vad Ska Byggas

Ett komplett system som implementerar:

1. **Objektdatastruktur** - Minimalistiska objekt med endast 4 attribut
2. **Metadata-arkitektur** - Flexibel EAV-modell (Entity-Attribute-Value)
3. **Nedärvningssystem** - Metadata som ärvs från förälder till barn i klusterträdet
4. **Korsbefruktning** - Metadata som kopplas till annan metadata
5. **Geografisk upplösningsordning** - Intelligent prioritering av positionsdata
6. **API-lager** - RESTful API för CRUD-operationer
7. **Frontend-komponenter** - React-komponenter för att visa och redigera metadata

1.3 Varför Det Är Viktigt

Problem med Nuvarande System:
- 🔴 **Tunga filer** - Varje export innehåller ALL metadata, även om man bara vill ha objektlistan
- 🔴 **Svårt att underhålla** - Ny metadatatyp = ny kolumn i databasen
- 🔴 **Svårt att söka** - Metadata är "inbakad" i breda kolumner
- 🔴 **Svårt att filtrera** - Kan inte enkelt visa "bara objekt med GPS" eller "bara objekt med foton"
- 🔴 **Dålig skalbarhet** - Systemet blir långsammare med fler objekt och metadata

Fördelar med Nya Systemet:
- ✅ **Lätta filer** - Exportera bara objektdata om det är allt du behöver
- ✅ **Flexibelt** - Lägg till nya metadatatyper utan databasändringar
- ✅ **Sökbart** - Enkelt hitta objekt med specifik metadata
- ✅ **Filtrerbart** - Visa bara objekt med viss metadata
- ✅ **Skalbart** - Prestanda påverkas inte negativt av många metadatatyper
- ✅ **Återanvändbart** - Metadata kan delas mellan objekt via nedärvning

1.4 Framgångskriterier

✅ **Funktionella Kriterier:**
1. Objekt kan skapas med endast 4 attribut
2. Metadata kan kopplas till objekt dynamiskt
3. Metadata ärvs från förälder till barn enligt regler
4. Metadata kan kopplas till annan metadata (korsbefruktning)
5. Geografisk data har korrekt prioriteringsordning
6. API fungerar för alla CRUD-operationer
7. Frontend visar både lokala och ärvda metadata tydligt

✅ **Prestanda Kriterier:**
1. Hämta objekt med all metadata < 200ms (inklusive ärvd)
2. Skapa/uppdatera metadata < 100ms
3. Trädsökning (hitta alla barn) < 300ms för 1000 objekt
4. Export av objektdata (utan metadata) < 50ms för 1000 objekt

✅ **Kvalitetskriterier:**
1. Datavalidering fungerar för alla metadatatyper
2. Ärvningslogik är korrekt implementerad
3. Inga data-inconsistencies
4. Fullständig error handling
5. Enhetstester för alla kritiska funktioner

Del 2: Databasimplementation

2.1 Schema - Drizzle ORM

Installation

```
npm install drizzle-orm postgres
npm install -D drizzle-kit
```

Konfiguration: `drizzle.config.ts`

```
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
  schema: './src/db/schema.ts',
  out: './drizzle',
  dialect: 'postgresql',
  dbCredentials: {
    url: process.env.DATABASE_URL!,
  },
});
```

Schema: `src/db/schema.ts`

```
import {
  pgTable,
  bigserial,
  varchar,
  text,
  integer,
  decimal,
  boolean,
  timestamp,
  json,
  index,
  foreignKey,
  pgEnum,
  bigint,
  check
} from 'drizzle-orm/pg-core';
import { relations } from 'drizzle-orm';

// ==============================================================================
// ENUMS
// ==============================================================================

export const objektTypEnum = pgEnum('objekt_typ', ['organisatorisk', 'fysisk']);

export const metadataDataTypEnum = pgEnum('metadata_datatyp', [
  'string',
  'integer',
  'decimal',
  'boolean',
  'datetime',
  'json',
  'referens'
]);

// ==============================================================================
// TABELLER
// ==============================================================================

// ------------------------------------------------------------------------------
// OBJEKT - Kärnentitet
// ------------------------------------------------------------------------------
export const clusterObjects = pgTable('cluster_objects', {
  id: bigserial('id', { mode: 'number' }).primaryKey(),
  namn: varchar('namn', { length: 255 }).notNull(),
  typ: objektTypEnum('typ').notNull(),
  parentId: bigint('parent_id', { mode: 'number' }),

  // Tidsstämplar
  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
  skapadAv: varchar('skapad_av', { length: 100 }),
  uppdateradDatum: timestamp('uppdaterad_datum').defaultNow().notNull(),
  uppdateradAv: varchar('uppdaterad_av', { length: 100 }),
}, (table) => ({
  // Index för prestanda
  parentIdx: index('idx_cluster_objects_parent').on(table.parentId),
  typIdx: index('idx_cluster_objects_typ').on(table.typ),
  namnIdx: index('idx_cluster_objects_namn').on(table.namn),

  // Foreign key till sig själv för parent-relationen
  parentFk: foreignKey({
    columns: [table.parentId],
    foreignColumns: [table.id],
```

```
    name: 'fk_cluster_objects_parent'
  }).onDelete('restrict'),
}));

// ---------------------------------------------------------------------------
// METADATATYPER - Katalog över tillåtna metadatatyper
// ---------------------------------------------------------------------------
export const metadataTypes = pgTable('metadata_types', {
  id: integer('id').primaryKey().generatedAlwaysAsIdentity(),
  namn: varchar('namn', { length: 100 }).notNull().unique(),
  beskrivning: text('beskrivning'),
  datatyp: metadataDataTypEnum('datatyp').notNull(),

  // För referens-datatyper: vilken tabell pekar de på?
  referensTabell: varchar('referens_tabell', { length: 100 }),

  // Är denna metadata logisk (används i systemlogik) eller ologisk (bara info)?
  arLogisk: boolean('ar_logisk').default(true).notNull(),

  // Standardvärde för om metadata ska ärvas
  standardArvs: boolean('standard_arvs').default(false).notNull(),

  // Tidsstämplar
  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
});

// ---------------------------------------------------------------------------
// METADATA - Faktiska metadata-värden
// ---------------------------------------------------------------------------
export const metadata = pgTable('metadata', {
  id: bigserial('id', { mode: 'number' }).primaryKey(),
  objektId: bigint('objekt_id', { mode: 'number' }).notNull(),
  metadatatypId: integer('metadatatyp_id').notNull(),

  // Värdefält - endast ett ska ha värde baserat på datatyp
  vardeString: text('varde_string'),
  vardeInteger: bigint('varde_integer', { mode: 'number' }),
  vardeDecimal: decimal('varde_decimal', { precision: 15, scale: 5 }),
  vardeBoolean: boolean('varde_boolean'),
  vardeDatetime: timestamp('varde_datetime'),
  vardeJson: json('varde_json'),
  vardeReferens: varchar('varde_referens', { length: 100 }),

  // Ärvning
  arvsNedåt: boolean('arvs_nedat').default(false).notNull(),
  stoppaVidareArvning: boolean('stoppa_vidare_arvning').default(false).notNull(),

  // Korsbefruktning - kan denna metadata kopplas till annan metadata?
  kopplad TillMetadataId: bigint('kopplad_till_metadata_id', { mode: 'number' }),

  // Tidsstämplar
  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
  skapadAv: varchar('skapad_av', { length: 100 }),
  uppdateradDatum: timestamp('uppdaterad_datum').defaultNow().notNull(),
  uppdateradAv: varchar('uppdaterad_av', { length: 100 }),
}, (table) => ({
  // Index
  objektIdx: index('idx_metadata_objekt').on(table.objektId),
  typIdx: index('idx_metadata_typ').on(table.metadatatypId),
  kopplingIdx: index('idx_metadata_koppling').on(table.kopplad TillMetadataId),
  objektTypIdx: index('idx_metadata_objekt_typ').on(table.objektId, table.metadatatypI
d),
```

```
  // Foreign keys
  objektFk: foreignKey({
    columns: [table.objektId],
    foreignColumns: [clusterObjects.id],
    name: 'fk_metadata_objekt'
  }).onDelete('cascade'),

  typFk: foreignKey({
    columns: [table.metadatatypId],
    foreignColumns: [metadataTypes.id],
    name: 'fk_metadata_typ'
  }).onDelete('restrict'),

  kopplingFk: foreignKey({
    columns: [table.koppladTillMetadataId],
    foreignColumns: [table.id],
    name: 'fk_metadata_koppling'
  }).onDelete('set null'),
}));

// ==============================================================================
// RELATIONER (Drizzle ORM)
// ==============================================================================

export const clusterObjectsRelations = relations(clusterObjects, ({ one, many }) => ({
  // Förälder-relation
  parent: one(clusterObjects, {
    fields: [clusterObjects.parentId],
    references: [clusterObjects.id],
    relationName: 'parent_child'
  }),

  // Barn-relation
  children: many(clusterObjects, {
    relationName: 'parent_child'
  }),

  // Metadata-relation
  metadata: many(metadata),
}));

export const metadataRelations = relations(metadata, ({ one, many }) => ({
  // Objekt som metadatan tillhör
  objekt: one(clusterObjects, {
    fields: [metadata.objektId],
    references: [clusterObjects.id],
  }),

  // Typ av metadata
  typ: one(metadataTypes, {
    fields: [metadata.metadatatypId],
    references: [metadataTypes.id],
  }),

  // Koppling till annan metadata (korsbefruktning)
  koppladTillMetadata: one(metadata, {
    fields: [metadata.koppladTillMetadataId],
    references: [metadata.id],
    relationName: 'metadata_korsbefruktar'
  }),

  // Metadata som är kopplade till denna metadata
  koppladeFran: many(metadata, {
```

```
    relationName: 'metadata_korsbefruktar'
  }),
}));

export const metadataTypesRelations = relations(metadataTypes, ({ many }) => ({
  metadata: many(metadata),
}));

// ==============================================================================
// SIDOREGISTER
// ==============================================================================

// ------------------------------------------------------------------------------
// KUNDER
// ------------------------------------------------------------------------------
export const customers = pgTable('customers', {
  id: integer('id').primaryKey().generatedAlwaysAsIdentity(),
  kundnummer: varchar('kundnummer', { length: 50 }).notNull().unique(),
  namn: varchar('namn', { length: 255 }).notNull(),
  organisationsnummer: varchar('organisationsnummer', { length: 20 }),
  adress: text('adress'),

  // Referenser till andra register
  avtalsId: integer('avtals_id'),
  prislistaId: integer('prislista_id'),

  // Tidsstämplar
  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
  uppdateradDatum: timestamp('uppdaterad_datum').defaultNow().notNull(),
}, (table) => ({
  kundnummerIdx: index('idx_customers_kundnummer').on(table.kundnummer),
}));

// ------------------------------------------------------------------------------
// ARTIKLAR
// ------------------------------------------------------------------------------
export const artikelTypEnum = pgEnum('artikel_typ', ['tjänst', 'vara', 'struktur-
artikel']);
export const sparrTypEnum = pgEnum('sparr_typ', ['ingen', 'per_adress', 'per_objekt'])
;

export const articles = pgTable('articles', {
  id: integer('id').primaryKey().generatedAlwaysAsIdentity(),
  artikelnummer: varchar('artikelnummer', { length: 50 }).notNull().unique(),
  namn: varchar('namn', { length: 255 }).notNull(),
  beskrivning: text('beskrivning'),
  typ: artikelTypEnum('typ').notNull(),

  // De tre vikterna
  forsaljningspris: decimal('forsaljningspris', { precision: 10, scale: 2 }),
  kostnad: decimal('kostnad', { precision: 10, scale: 2 }),
  tidEllerLedtid: integer('tid_eller_ledtid'), // Minuter för tjänst, dagar för vara

  // Multiplikation och spärrar
  kanMultipliceras: boolean('kan_multipliceras').default(true).notNull(),
  sparrTyp: sparrTypEnum('sparr_typ').default('ingen').notNull(),

  // Tidsstämplar
  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
  uppdateradDatum: timestamp('uppdaterad_datum').defaultNow().notNull(),
}, (table) => ({
  artikelnummerIdx: index('idx_articles_artikelnummer').on(table.artikelnummer),
  typIdx: index('idx_articles_typ').on(table.typ),
```

```
}));

// ---------------------------------------------------------------------------
// PRISLISTOR
// ---------------------------------------------------------------------------
export const prislistaTypEnum = pgEnum('prislista_typ', [
  'standard',
  'kund',
  'sason',
  'mangd',
  'kedja'
]);

export const pricelists = pgTable('pricelists', {
  id: integer('id').primaryKey().generatedAlwaysAsIdentity(),
  namn: varchar('namn', { length: 255 }).notNull(),
  giltigFran: timestamp('giltig_fran'),
  giltigTill: timestamp('giltig_till'),
  typ: prislistaTypEnum('typ').default('standard').notNull(),

  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
  uppdateradDatum: timestamp('uppdaterad_datum').defaultNow().notNull(),
});

export const pricelistPrices = pgTable('pricelist_prices', {
  id: bigserial('id', { mode: 'number' }).primaryKey(),
  prislistaId: integer('prislista_id').notNull(),
  artikelId: integer('artikel_id').notNull(),
  pris: decimal('pris', { precision: 10, scale: 2 }).notNull(),

  // För mängdbaserade prislistor
  franAntal: integer('fran_antal').default(1).notNull(),
  tillAntal: integer('till_antal').default(999999).notNull(),

  skapadDatum: timestamp('skapad_datum').defaultNow().notNull(),
}, (table) => ({
  prislistaFk: foreignKey({
    columns: [table.prislistaId],
    foreignColumns: [pricelists.id],
    name: 'fk_pricelist_prices_prislista'
  }).onDelete('cascade'),

  artikelFk: foreignKey({
    columns: [table.artikelId],
    foreignColumns: [articles.id],
    name: 'fk_pricelist_prices_artikel'
  }).onDelete('cascade'),

  // Unikt index för att undvika dubbletter
  uniquePris: index('idx_pricelist_prices_unique').on(
    table.prislistaId,
    table.artikelId,
    table.franAntal
  ),
}));

// ===========================================================================
// TYPESCRIPT TYPES (för användning i applikationen)
// ===========================================================================

export type ClusterObject = typeof clusterObjects.$inferSelect;
export type NewClusterObject = typeof clusterObjects.$inferInsert;
```

```
export type Metadata = typeof metadata.$inferSelect;
export type NewMetadata = typeof metadata.$inferInsert;

export type MetadataType = typeof metadataTypes.$inferSelect;
export type NewMetadataType = typeof metadataTypes.$inferInsert;

export type Customer = typeof customers.$inferSelect;
export type NewCustomer = typeof customers.$inferInsert;

export type Article = typeof articles.$inferSelect;
export type NewArticle = typeof articles.$inferInsert;

export type Pricelist = typeof pricelists.$inferSelect;
export type NewPricelist = typeof pricelists.$inferInsert;

export type PricelistPrice = typeof pricelistPrices.$inferSelect;
export type NewPricelistPrice = typeof pricelistPrices.$inferInsert;
```

2.2 Migrationer

Databas-setup: `src/db/index.ts`

```
import { drizzle } from 'drizzle-orm/postgres-js';
import postgres from 'postgres';
import * as schema from './schema';

const connectionString = process.env.DATABASE_URL!;

// För queries
const client = postgres(connectionString);
export const db = drizzle(client, { schema });

// För migrationer
export const migrationClient = postgres(connectionString, { max: 1 });
```

Skapa migrering

```
# Generera migrering baserat på schema
npx drizzle-kit generate:pg

# Kör migrering
npx drizzle-kit push:pg
```

Manuell migration (om behövs): `drizzle/0000_initial.sql`

```sql
-- ENUMS
CREATE TYPE objekt_typ AS ENUM ('organisatorisk', 'fysisk');
CREATE TYPE metadata_datatyp AS ENUM ('string', 'integer', 'decimal', 'boolean', 'dat-
etime', 'json', 'referens');
CREATE TYPE artikel_typ AS ENUM ('tjänst', 'vara', 'strukturartikel');
CREATE TYPE sparr_typ AS ENUM ('ingen', 'per_adress', 'per_objekt');
CREATE TYPE prislista_typ AS ENUM ('standard', 'kund', 'sason', 'mangd', 'kedja');

-- CLUSTER_OBJECTS
CREATE TABLE cluster_objects (
    id BIGSERIAL PRIMARY KEY,
    namn VARCHAR(255) NOT NULL,
    typ objekt_typ NOT NULL,
    parent_id BIGINT,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    skapad_av VARCHAR(100),
    uppdaterad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    uppdaterad_av VARCHAR(100),
    CONSTRAINT fk_cluster_objects_parent FOREIGN KEY (parent_id)
        REFERENCES cluster_objects(id) ON DELETE RESTRICT
);

CREATE INDEX idx_cluster_objects_parent ON cluster_objects(parent_id);
CREATE INDEX idx_cluster_objects_typ ON cluster_objects(typ);
CREATE INDEX idx_cluster_objects_namn ON cluster_objects(namn);

-- METADATA_TYPES
CREATE TABLE metadata_types (
    id SERIAL PRIMARY KEY,
    namn VARCHAR(100) UNIQUE NOT NULL,
    beskrivning TEXT,
    datatyp metadata_datatyp NOT NULL,
    referens_tabell VARCHAR(100),
    ar_logisk BOOLEAN DEFAULT TRUE NOT NULL,
    standard_arvs BOOLEAN DEFAULT FALSE NOT NULL,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- METADATA
CREATE TABLE metadata (
    id BIGSERIAL PRIMARY KEY,
    objekt_id BIGINT NOT NULL,
    metadatatyp_id INTEGER NOT NULL,
    varde_string TEXT,
    varde_integer BIGINT,
    varde_decimal DECIMAL(15, 5),
    varde_boolean BOOLEAN,
    varde_datetime TIMESTAMP,
    varde_json JSON,
    varde_referens VARCHAR(100),
    arvs_nedat BOOLEAN DEFAULT FALSE NOT NULL,
    stoppa_vidare_arvning BOOLEAN DEFAULT FALSE NOT NULL,
    kopplad_till_metadata_id BIGINT,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    skapad_av VARCHAR(100),
    uppdaterad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    uppdaterad_av VARCHAR(100),
    CONSTRAINT fk_metadata_objekt FOREIGN KEY (objekt_id)
        REFERENCES cluster_objects(id) ON DELETE CASCADE,
    CONSTRAINT fk_metadata_typ FOREIGN KEY (metadatatyp_id)
        REFERENCES metadata_types(id) ON DELETE RESTRICT,
    CONSTRAINT fk_metadata_koppling FOREIGN KEY (kopplad_till_metadata_id)
```

```sql
        REFERENCES metadata(id) ON DELETE SET NULL
);

CREATE INDEX idx_metadata_objekt ON metadata(objekt_id);
CREATE INDEX idx_metadata_typ ON metadata(metadatatyp_id);
CREATE INDEX idx_metadata_koppling ON metadata(kopplad_till_metadata_id);
CREATE INDEX idx_metadata_objekt_typ ON metadata(objekt_id, metadatatyp_id);

-- CUSTOMERS
CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    kundnummer VARCHAR(50) UNIQUE NOT NULL,
    namn VARCHAR(255) NOT NULL,
    organisationsnummer VARCHAR(20),
    adress TEXT,
    avtals_id INTEGER,
    prislista_id INTEGER,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    uppdaterad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE INDEX idx_customers_kundnummer ON customers(kundnummer);

-- ARTICLES
CREATE TABLE articles (
    id SERIAL PRIMARY KEY,
    artikelnummer VARCHAR(50) UNIQUE NOT NULL,
    namn VARCHAR(255) NOT NULL,
    beskrivning TEXT,
    typ artikel_typ NOT NULL,
    forsaljningspris DECIMAL(10, 2),
    kostnad DECIMAL(10, 2),
    tid_eller_ledtid INTEGER,
    kan_multipliceras BOOLEAN DEFAULT TRUE NOT NULL,
    sparr_typ sparr_typ DEFAULT 'ingen' NOT NULL,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    uppdaterad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE INDEX idx_articles_artikelnummer ON articles(artikelnummer);
CREATE INDEX idx_articles_typ ON articles(typ);

-- PRICELISTS
CREATE TABLE pricelists (
    id SERIAL PRIMARY KEY,
    namn VARCHAR(255) NOT NULL,
    giltig_fran TIMESTAMP,
    giltig_till TIMESTAMP,
    typ prislista_typ DEFAULT 'standard' NOT NULL,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    uppdaterad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- PRICELIST_PRICES
CREATE TABLE pricelist_prices (
    id BIGSERIAL PRIMARY KEY,
    prislista_id INTEGER NOT NULL,
    artikel_id INTEGER NOT NULL,
    pris DECIMAL(10, 2) NOT NULL,
    fran_antal INTEGER DEFAULT 1 NOT NULL,
    till_antal INTEGER DEFAULT 999999 NOT NULL,
    skapad_datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_pricelist_prices_prislista FOREIGN KEY (prislista_id)
```

```
        REFERENCES pricelists(id) ON DELETE CASCADE,
    CONSTRAINT fk_pricelist_prices_artikel FOREIGN KEY (artikel_id)
        REFERENCES articles(id) ON DELETE CASCADE
);

CREATE INDEX idx_pricelist_prices_unique ON pricelist_prices(prislista_id,
artikel_id, fran_antal);
```

Seed Data: `src/db/seed.ts`

```
import { db } from './index';
import {
  clusterObjects,
  metadataTypes,
  metadata,
  customers,
  articles,
  pricelists
} from './schema';

async function seed() {
  console.log('🌱 Seeding database...');

  // ============================================================================
  // 1. METADATA-TYPER
  // ============================================================================
  console.log('📝 Creating metadata types...');

  const metadataTypesData: any[] = [
    // Geografiska metadata
    { namn: 'Adress', datatyp: 'string', arLogisk: true, standardArvs: true,
      beskrivning: 'Postadress (grov position)' },
    { namn: 'GPS', datatyp: 'string', arLogisk: true, standardArvs: true,
      beskrivning: 'GPS-koordinater (longitud, latitud)' },
    { namn: 'What3words', datatyp: 'string', arLogisk: true, standardArvs: true,
      beskrivning: 'What3words adress (3x3 meter precision)' },

    // Kvantitativa metadata
    { namn: 'Antal', datatyp: 'integer', arLogisk: true, standardArvs: false,
      beskrivning: 'Antal av objektet' },
    { namn: 'Area', datatyp: 'decimal', arLogisk: true, standardArvs: false,
      beskrivning: 'Storlek i kvadratmeter' },
    { namn: 'Volym', datatyp: 'integer', arLogisk: true, standardArvs: false,
      beskrivning: 'Volym i liter' },

    // Administrativa metadata
    { namn: 'Kund', datatyp: 'referens', referensTabell: 'customers',
      arLogisk: true, standardArvs: true, beskrivning: 'Kund-referens' },
    { namn: 'Kundnummer', datatyp: 'string', arLogisk: true, standardArvs: true,
      beskrivning: 'Kundnummer' },
    { namn: 'Er_Referens', datatyp: 'string', arLogisk: true, standardArvs: false,
      beskrivning: 'Kundens referens' },
    { namn: 'Er_Ordernummer', datatyp: 'string', arLogisk: true, standardArvs: false,
      beskrivning: 'Kundens ordernummer' },

    // Kontaktmetadata
    { namn: 'Kontaktperson_Namn', datatyp: 'string', arLogisk: true, standardArvs: tru
e,
      beskrivning: 'Kontaktpersonens namn' },
    { namn: 'Kontaktperson_Telefon', datatyp: 'string', arLogisk: true, standardArvs:
true,
      beskrivning: 'Kontaktpersonens telefonnummer' },
    { namn: 'Kontaktperson_Epost', datatyp: 'string', arLogisk: true, standardArvs: tr
ue,
      beskrivning: 'Kontaktpersonens e-post' },
    { namn: 'Kontaktperson_Roll', datatyp: 'string', arLogisk: true, standardArvs: tru
e,
      beskrivning: 'Kontaktpersonens roll (t.ex. Bovärd, Förvaltare)' },

    // Beskrivande metadata
    { namn: 'Beskrivning', datatyp: 'string', arLogisk: false, standardArvs: false,
      beskrivning: 'Fritextbeskrivning' },
```

```
  { namn: 'Anteckningar', datatyp: 'string', arLogisk: false, standardArvs: false,
    beskrivning: 'Praktiska anteckningar' },
  { namn: 'Markning', datatyp: 'string', arLogisk: false, standardArvs: false,
    beskrivning: 'Taggar/etiketter' },

  // Artikelkopplad metadata
  { namn: 'Artikel', datatyp: 'referens', referensTabell: 'articles',
    arLogisk: true, standardArvs: false, beskrivning: 'Artikel-referens' },
  { namn: 'Prislista', datatyp: 'referens', referensTabell: 'pricelists',
    arLogisk: true, standardArvs: true, beskrivning: 'Prislista-referens' },

  // Tidsrelaterad metadata
  { namn: 'Frekvens', datatyp: 'string', arLogisk: true, standardArvs: false,
    beskrivning: 'Hur ofta något ska göras' },
  { namn: 'Tidsfonster', datatyp: 'json', arLogisk: true, standardArvs: true,
    beskrivning: 'När något får/måste göras' },

  // Detaljtyp (vad för slags fysiskt/organisatoriskt objekt)
  { namn: 'Detaljtyp', datatyp: 'string', arLogisk: true, standardArvs: false,
    beskrivning: 'Specifik typ av objekt (t.ex. Pantkärl_160L, Miljörum)' },

  // Kodåtkomst
  { namn: 'Kod', datatyp: 'string', arLogisk: true, standardArvs: true,
    beskrivning: 'Åtkomstkod till objekt/område' },

  // Rating
  { namn: 'Rating', datatyp: 'integer', arLogisk: true, standardArvs: false,
    beskrivning: 'Betyg/rating (t.ex. 4 av 5)' },

  // Bilagor
  { namn: 'Foto', datatyp: 'json', arLogisk: false, standardArvs: false,
    beskrivning: 'Foton kopplade till objektet' },
  { namn: 'Filer', datatyp: 'json', arLogisk: false, standardArvs: false,
    beskrivning: 'Dokument/filer kopplade till objektet' },
];

await db.insert(metadataTypes).values(metadataTypesData);
console.log(`✅ Created ${metadataTypesData.length} metadata types`);

// ============================================================================
// 2. KUNDER
// ============================================================================
console.log('👥 Creating customers...');

const [axfood] = await db.insert(customers).values({
  kundnummer: 'KUND_AXFOOD',
  namn: 'Axfood',
  organisationsnummer: '556004-8680',
  adress: 'Hemköpsvägen 6, 171 93 Solna',
}).returning();

const [brfTornet] = await db.insert(customers).values({
  kundnummer: 'KUND_BRF_TORNET_4',
  namn: 'BRF Tornet 4',
  organisationsnummer: '716400-1234',
  adress: 'Tongatan 22, Uppsala',
}).returning();

console.log(`✅ Created 2 customers`);

// ============================================================================
// 3. ARTIKLAR
// ============================================================================
```

```
console.log('📦 Creating articles...');

const [tvattaPantkarlArtikel] = await db.insert(articles).values({
  artikelnummer: 'ART_TVATTA_PANTKARL',
  namn: 'Tvätta pantkärl',
  beskrivning: 'Tvätta och rengöra pantkärl',
  typ: 'tjänst',
  forsaljningspris: '50.00',
  kostnad: '25.00',
  tidEllerLedtid: 5, // 5 minuter per pantkärl
  kanMultipliceras: true,
  sparrTyp: 'ingen',
}).returning();

const [rumstvattArtikel] = await db.insert(articles).values({
  artikelnummer: 'ART_RUMSTVATT',
  namn: 'Rumstvätt',
  beskrivning: 'Tvätta golv och väggar i rum',
  typ: 'tjänst',
  forsaljningspris: '500.00',
  kostnad: '250.00',
  tidEllerLedtid: 30, // 30 minuter
  kanMultipliceras: false,
  sparrTyp: 'per_objekt',
}).returning();

const [etableringArtikel] = await db.insert(articles).values({
  artikelnummer: 'ART_ETABLERING',
  namn: 'Etablering',
  beskrivning: 'Framkörningsavgift / etablering på platsen',
  typ: 'tjänst',
  forsaljningspris: '300.00',
  kostnad: '150.00',
  tidEllerLedtid: 15, // 15 minuter
  kanMultipliceras: false,
  sparrTyp: 'per_adress',
}).returning();

console.log(`✅ Created 3 articles`);

// ============================================================================
// 4. PRISLISTA
// ============================================================================
console.log('💰 Creating pricelist...');

const [standardPrislista] = await db.insert(pricelists).values({
  namn: 'Standard 2026',
  typ: 'standard',
  giltigFran: new Date('2026-01-01'),
  giltigTill: new Date('2026-12-31'),
}).returning();

console.log(`✅ Created pricelist`);

// ============================================================================
// 5. KLUSTERSTRUKTUR - Willys Södermalm Exempel
// ============================================================================
console.log('🏢 Creating cluster structure...');

// Nivå 1: Axfood koncern
const [axfoodKoncern] = await db.insert(clusterObjects).values({
  namn: 'Axfood',
  typ: 'organisatorisk',
```

```
    parentId: null,
  }).returning();

  // Nivå 2: Willys kedja
  const [willysKedja] = await db.insert(clusterObjects).values({
    namn: 'Willys',
    typ: 'organisatorisk',
    parentId: axfoodKoncern.id,
  }).returning();

  // Nivå 3: Distrikt Stockholm
  const [distriktStockholm] = await db.insert(clusterObjects).values({
    namn: 'Distrikt Stockholm',
    typ: 'organisatorisk',
    parentId: willysKedja.id,
  }).returning();

  // Nivå 4: Willys Södermalm (butik)
  const [willysSodermalm] = await db.insert(clusterObjects).values({
    namn: 'Willys Södermalm',
    typ: 'organisatorisk',
    parentId: distriktStockholm.id,
  }).returning();

  // Nivå 5: Pantrum (område)
  const [pantrum] = await db.insert(clusterObjects).values({
    namn: 'Pantrum',
    typ: 'organisatorisk',
    parentId: willysSodermalm.id,
  }).returning();

  // Nivå 6: Fysiska objekt i pantrum
  const [pantkarl] = await db.insert(clusterObjects).values({
    namn: 'Pantkärl',
    typ: 'fysisk',
    parentId: pantrum.id,
  }).returning();

  const [pantmaskin] = await db.insert(clusterObjects).values({
    namn: 'Pantmaskin',
    typ: 'fysisk',
    parentId: pantrum.id,
  }).returning();

  const [golvbrunn] = await db.insert(clusterObjects).values({
    namn: 'Golvbrunn',
    typ: 'fysisk',
    parentId: pantrum.id,
  }).returning();

  console.log(`✅ Created cluster structure with ${8} objects`);

  // ============================================================================
  // 6. METADATA PÅ WILLYS SÖDERMALM
  // ============================================================================
  console.log('🏷️  Adding metadata to Willys Södermalm...');

  // Hämta metadata-typer
  const metaTypes = await db.select().from(metadataTypes);
  const getMetaTypeId = (namn: string) =>
    metaTypes.find(t => t.namn === namn)?.id!;

  await db.insert(metadata).values([
```

```
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Kund'),
      vardeReferens: axfood.kundnummer,
      arvsNedat: true,
    },
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Adress'),
      vardeString: 'Götgatan 100, Stockholm',
      arvsNedat: true,
    },
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Tidsfonster'),
      vardeJson: {
        dag: 'Måndag-Fredag',
        tid: '06:00-10:00',
        orsak: 'Butik öppnar 10:00',
      },
      arvsNedat: true,
    },
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Kontaktperson_Namn'),
      vardeString: 'Lisa Andersson',
      arvsNedat: true,
    },
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Kontaktperson_Telefon'),
      vardeString: '070-123 45 67',
      arvsNedat: true,
    },
    {
      objektId: willysSodermalm.id,
      metadatatypId: getMetaTypeId('Kontaktperson_Roll'),
      vardeString: 'Butikschef',
      arvsNedat: true,
    },
  ]);

  console.log(`✅ Added metadata to Willys Södermalm`);

  // ============================================================================
  // 7. METADATA PÅ PANTRUM
  // ============================================================================
  console.log('🏷️  Adding metadata to Pantrum...');

  await db.insert(metadata).values([
    {
      objektId: pantrum.id,
      metadatatypId: getMetaTypeId('GPS'),
      vardeString: '59.31563, 18.06493',
      arvsNedat: true,
    },
    {
      objektId: pantrum.id,
      metadatatypId: getMetaTypeId('What3words'),
      vardeString: 'lamps.divide.soccer',
      arvsNedat: true,
    },
    {
```

```
      objektId: pantrum.id,
      metadatatypId: getMetaTypeId('Kod'),
      vardeString: '1234',
      arvsNedat: true,
    },
    {
      objektId: pantrum.id,
      metadatatypId: getMetaTypeId('Detaljtyp'),
      vardeString: 'Pantrum',
      arvsNedat: false,
    },
  ]);

  console.log(`✅ Added metadata to Pantrum`);

  // ============================================================================
  // 8. METADATA PÅ PANTKÄRL (inkl. korsbefruktning)
  // ============================================================================
  console.log('🏷️  Adding metadata to Pantkärl (with cross-fertilization)...');

  // Lägg till metadata för detaljtyp
  await db.insert(metadata).values({
    objektId: pantkarl.id,
    metadatatypId: getMetaTypeId('Detaljtyp'),
    vardeString: 'Pantkärl_160L',
    arvsNedat: false,
  });

  // Lägg till artikel-metadata
  const [artikelMetadata] = await db.insert(metadata).values({
    objektId: pantkarl.id,
    metadatatypId: getMetaTypeId('Artikel'),
    vardeReferens: tvattaPantkarlArtikel.artikelnummer,
    arvsNedat: false,
  }).returning();

  // Lägg till antal - KOPPLAD TILL ARTIKEL (korsbefruktning!)
  await db.insert(metadata).values({
    objektId: pantkarl.id,
    metadatatypId: getMetaTypeId('Antal'),
    vardeInteger: 10,
    arvsNedat: false,
    koppladTillMetadataId: artikelMetadata.id, // ← KORSBEFRUKTNING!
  });

  // Lägg till rating - KOPPLAD TILL ARTIKEL (korsbefruktning!)
  await db.insert(metadata).values({
    objektId: pantkarl.id,
    metadatatypId: getMetaTypeId('Rating'),
    vardeInteger: 4,
    arvsNedat: false,
    koppladTillMetadataId: artikelMetadata.id, // ← KORSBEFRUKTNING!
  });

  console.log(`✅ Added metadata to Pantkärl with cross-fertilization`);

  // ============================================================================
  // 9. BRF TORNET 4 EXEMPEL
  // ============================================================================
  console.log('🏢 Creating BRF Tornet 4 structure...');

  // Nivå 1: BRF Tornet 4
  const [brfTornet4] = await db.insert(clusterObjects).values({
```

```
    namn: 'BRF Tornet 4',
    typ: 'organisatorisk',
    parentId: null,
  }).returning();

  // Nivå 2: Tongatan 22
  const [tongatan22] = await db.insert(clusterObjects).values({
    namn: 'Tongatan 22',
    typ: 'organisatorisk',
    parentId: brfTornet4.id,
  }).returning();

  // Nivå 3: Miljörum
  const [miljorum] = await db.insert(clusterObjects).values({
    namn: 'Miljörum',
    typ: 'fysisk',
    parentId: tongatan22.id,
  }).returning();

  // Metadata på BRF Tornet 4
  await db.insert(metadata).values([
    {
      objektId: brfTornet4.id,
      metadatatypId: getMetaTypeId('Kund'),
      vardeReferens: brfTornet.kundnummer,
      arvsNedat: true,
    },
    {
      objektId: brfTornet4.id,
      metadatatypId: getMetaTypeId('Adress'),
      vardeString: 'Tongatan 22, Uppsala',
      arvsNedat: true,
    },
  ]);

  // Metadata på Miljörum
  await db.insert(metadata).values([
    {
      objektId: miljorum.id,
      metadatatypId: getMetaTypeId('Detaljtyp'),
      vardeString: 'Miljörum',
      arvsNedat: false,
    },
    {
      objektId: miljorum.id,
      metadatatypId: getMetaTypeId('GPS'),
      vardeString: '59.85882, 17.63861',
      arvsNedat: true,
    },
  ]);

  console.log(`✅ Created BRF Tornet 4 structure`);

  console.log('\n🎉 Seeding completed successfully!');
}

// Kör seed
seed()
  .catch((error) => {
    console.error('❌ Seeding failed:', error);
    process.exit(1);
  })
  .finally(() => {
```

```
    process.exit(0);
  });
```

Kör seed

```
# Lägg till script i package.json
"scripts": {
  "db:seed": "tsx src/db/seed.ts"
}

# Kör seed
npm run db:seed
```

2.3 Queries och Funktioner

Core Query Functions: `src/db/queries.ts`

```
import { db } from './index';
import { clusterObjects, metadata, metadataTypes, customers, articles } from './
schema';
import { eq, and, sql, inArray } from 'drizzle-orm';
import type { Metadata, MetadataType } from './schema';

// ==============================================================================
// INTERFACES
// ==============================================================================

export interface MetadataWithType extends Metadata {
  typ: MetadataType;
  fromObject?: {
    id: number;
    namn: string;
    level: number;
  };
  source: 'local' | 'inherited';
}

export interface ObjectWithMetadata {
  id: number;
  namn: string;
  typ: 'organisatorisk' | 'fysisk';
  parentId: number | null;
  metadata: MetadataWithType[];
}

export interface GeographicPosition {
  typ: 'GPS' | 'What3words' | 'Adress';
  precision: 'exakt' | 'medel' | 'grov';
  varde: string;
  fromObject?: {
    id: number;
    namn: string;
  };
}

// ==============================================================================
// 1. HÄMTA OBJEKT MED ALL METADATA (INKL. ÄRVD)
// ==============================================================================

export async function getObjectWithAllMetadata(objektId: number): Promise<ObjectWith-
Metadata | null> {
  // Hämta objektet
  const [objekt] = await db
    .select()
    .from(clusterObjects)
    .where(eq(clusterObjects.id, objektId));

  if (!objekt) return null;

  // Hämta parent chain med rekursiv CTE
  const parentChainQuery = sql`
    WITH RECURSIVE parent_chain AS (
      -- Starta med aktuellt objekt
      SELECT
        id,
        parent_id,
        namn,
        0 as level
      FROM cluster_objects
```

```
      WHERE id = ${objektId}

      UNION ALL

      -- Gå uppåt i hierarkin
      SELECT
        o.id,
        o.parent_id,
        o.namn,
        pc.level + 1
      FROM cluster_objects o
      INNER JOIN parent_chain pc ON o.id = pc.parent_id
    )
    SELECT
      m.id,
      m.objekt_id,
      m.metadatatyp_id,
      m.varde_string,
      m.varde_integer,
      m.varde_decimal,
      m.varde_boolean,
      m.varde_datetime,
      m.varde_json,
      m.varde_referens,
      m.arvs_nedat,
      m.stoppa_vidare_arvning,
      m.kopplad_till_metadata_id,
      m.skapad_datum,
      m.uppdaterad_datum,
      mt.id as typ_id,
      mt.namn as typ_namn,
      mt.datatyp as typ_datatyp,
      mt.referens_tabell as typ_referens_tabell,
      mt.ar_logisk as typ_ar_logisk,
      pc.level,
      pc.namn as from_objekt_namn,
      CASE
        WHEN m.objekt_id = ${objektId} THEN 'local'
        ELSE 'inherited'
      END as source
    FROM parent_chain pc
    INNER JOIN metadata m ON m.objekt_id = pc.id
    INNER JOIN metadata_types mt ON m.metadatatyp_id = mt.id
    WHERE
      (m.objekt_id = ${objektId} OR m.arvs_nedat = TRUE)
    ORDER BY
      CASE WHEN m.objekt_id = ${objektId} THEN 0 ELSE 1 END,
      pc.level ASC
  `;

  const metadataResults = await db.execute(parentChainQuery);

  // Konvertera resultat till typad struktur
  const metadataWithTypes: MetadataWithType[] = metadataResults.rows.map((row: any)
=> ({
    id: row.id,
    objektId: row.objekt_id,
    metadatatypId: row.metadatatyp_id,
    vardeString: row.varde_string,
    vardeInteger: row.varde_integer ? Number(row.varde_integer) : null,
    vardeDecimal: row.varde_decimal,
    vardeBoolean: row.varde_boolean,
    vardeDatetime: row.varde_datetime,
```

```
    vardeJson: row.varde_json,
    vardeReferens: row.varde_referens,
    arvsNedat: row.arvs_nedat,
    stoppaVidareArvning: row.stoppa_vidare_arvning,
    koppladTillMetadataId: row.kopplad_till_metadata_id ? Number(row.kopplad_till_meta
data_id) : null,
    skapadDatum: row.skapad_datum,
    skapadAv: null,
    uppdateradDatum: row.uppdaterad_datum,
    uppdateradAv: null,
    typ: {
      id: row.typ_id,
      namn: row.typ_namn,
      datatyp: row.typ_datatyp,
      referensTabell: row.typ_referens_tabell,
      arLogisk: row.typ_ar_logisk,
      standardArvs: false,
      skapadDatum: new Date(),
      beskrivning: null,
    },
    source: row.source,
    fromObject: row.source === 'inherited' ? {
      id: row.objekt_id,
      namn: row.from_objekt_namn,
      level: row.level,
    } : undefined,
  }));

  return {
    ...objekt,
    metadata: metadataWithTypes,
  };
}

// ============================================================================
// 2. HÄMTA METADATA-VÄRDE (med ärvning)
// ============================================================================

export async function getMetadataValue(
  objektId: number,
  metadataTypNamn: string
): Promise<any | null> {
  const objectWithMetadata = await getObjectWithAllMetadata(objektId);

  if (!objectWithMetadata) return null;

  // Hitta metadata av rätt typ (lokalt först, sedan ärvt)
  const metadata = objectWithMetadata.metadata.find(m => m.typ.namn === metadataTyp-
Namn);

  if (!metadata) return null;

  // Returnera rätt värdefält baserat på datatyp
  switch (metadata.typ.datatyp) {
    case 'string':
      return metadata.vardeString;
    case 'integer':
      return metadata.vardeInteger;
    case 'decimal':
      return metadata.vardeDecimal;
    case 'boolean':
      return metadata.vardeBoolean;
    case 'datetime':
```

```
      return metadata.vardeDatetime;
    case 'json':
      return metadata.vardeJson;
    case 'referens':
      return metadata.vardeReferens;
    default:
      return null;
  }
}

// ==============================================================================
// 3. SKAPA METADATA
// ==============================================================================

export async function createMetadata(data: {
  objektId: number;
  metadataTypNamn: string;
  varde: any;
  arvsNedat?: boolean;
  koppladTillMetadataId?: number | null;
  skapadAv?: string;
}): Promise<Metadata> {
  // Hämta metadata-typ
  const [metadataTyp] = await db
    .select()
    .from(metadataTypes)
    .where(eq(metadataTypes.namn, data.metadataTypNamn));

  if (!metadataTyp) {
    throw new Error(`Metadata type "${data.metadataTypNamn}" not found`);
  }

  // Förbered värdefält baserat på datatyp
  const vardeFields: any = {
    vardeString: null,
    vardeInteger: null,
    vardeDecimal: null,
    vardeBoolean: null,
    vardeDatetime: null,
    vardeJson: null,
    vardeReferens: null,
  };

  switch (metadataTyp.datatyp) {
    case 'string':
      vardeFields.vardeString = String(data.varde);
      break;
    case 'integer':
      vardeFields.vardeInteger = parseInt(String(data.varde));
      if (isNaN(vardeFields.vardeInteger)) {
        throw new Error(`Invalid integer value: ${data.varde}`);
      }
      break;
    case 'decimal':
      vardeFields.vardeDecimal = String(parseFloat(String(data.varde)));
      if (isNaN(parseFloat(vardeFields.vardeDecimal))) {
        throw new Error(`Invalid decimal value: ${data.varde}`);
      }
      break;
    case 'boolean':
      vardeFields.vardeBoolean = Boolean(data.varde);
      break;
    case 'datetime':
```

```
      vardeFields.vardeDatetime = new Date(data.varde);
      if (isNaN(vardeFields.vardeDatetime.getTime())) {
        throw new Error(`Invalid datetime value: ${data.varde}`);
      }
      break;
    case 'json':
      vardeFields.vardeJson = typeof data.varde === 'string' ?
JSON.parse(data.varde) : data.varde;
      break;
    case 'referens':
      vardeFields.vardeReferens = String(data.varde);
      break;
  }

  // Skapa metadata
  const [newMetadata] = await db.insert(metadata).values({
    objektId: data.objektId,
    metadatatypId: metadataTyp.id,
    ...vardeFields,
    arvsNedat: data.arvsNedat ?? metadataTyp.standardArvs,
    koppladTillMetadataId: data.koppladTillMetadataId ?? null,
    skapadAv: data.skapadAv,
  }).returning();

  return newMetadata;
}

// ==============================================================================
// 4. UPPDATERA METADATA
// ==============================================================================

export async function updateMetadata(
  metadataId: number,
  varde: any,
  uppdateradAv?: string
): Promise<Metadata> {
  // Hämta existerande metadata
  const [existing] = await db
    .select()
    .from(metadata)
    .where(eq(metadata.id, metadataId));

  if (!existing) {
    throw new Error(`Metadata with id ${metadataId} not found`);
  }

  // Hämta metadata-typ för att validera
  const [metadataTyp] = await db
    .select()
    .from(metadataTypes)
    .where(eq(metadataTypes.id, existing.metadatatypId));

  // Förbered värdefält
  const vardeFields: any = {
    vardeString: null,
    vardeInteger: null,
    vardeDecimal: null,
    vardeBoolean: null,
    vardeDatetime: null,
    vardeJson: null,
    vardeReferens: null,
  };
```

```
  switch (metadataTyp.datatyp) {
    case 'string':
      vardeFields.vardeString = String(varde);
      break;
    case 'integer':
      vardeFields.vardeInteger = parseInt(String(varde));
      break;
    case 'decimal':
      vardeFields.vardeDecimal = String(parseFloat(String(varde)));
      break;
    case 'boolean':
      vardeFields.vardeBoolean = Boolean(varde);
      break;
    case 'datetime':
      vardeFields.vardeDatetime = new Date(varde);
      break;
    case 'json':
      vardeFields.vardeJson = typeof varde === 'string' ? JSON.parse(varde) : varde;
      break;
    case 'referens':
      vardeFields.vardeReferens = String(varde);
      break;
  }

  // Uppdatera
  const [updated] = await db
    .update(metadata)
    .set({
      ...vardeFields,
      uppdateradAv,
      uppdateradDatum: new Date(),
    })
    .where(eq(metadata.id, metadataId))
    .returning();

  return updated;
}

// ============================================================================
// 5. RADERA METADATA
// ============================================================================

export async function deleteMetadata(metadataId: number): Promise<void> {
  await db.delete(metadata).where(eq(metadata.id, metadataId));
}

// ============================================================================
// 6. HÄMTA KORSBEFRUKTAD METADATA
// ============================================================================

export async function getCrossFertilizedMetadata(
  objektId: number,
  baseMetadataTypNamn: string
): Promise<any[]> {
  const query = sql`
    SELECT
      m_base.id as base_id,
      mt_base.namn as base_typ,
      COALESCE(
        m_base.varde_string,
        CAST(m_base.varde_integer AS TEXT),
        CAST(m_base.varde_decimal AS TEXT),
        CAST(m_base.varde_boolean AS TEXT),
```

```
        m_base.varde_referens
      ) as base_varde,
      m_related.id as related_id,
      mt_related.namn as related_typ,
      COALESCE(
        m_related.varde_string,
        CAST(m_related.varde_integer AS TEXT),
        CAST(m_related.varde_decimal AS TEXT),
        CAST(m_related.varde_boolean AS TEXT),
        m_related.varde_referens
      ) as related_varde
    FROM metadata m_base
    INNER JOIN metadata_types mt_base ON m_base.metadatatyp_id = mt_base.id
    LEFT JOIN metadata m_related ON m_related.kopplad_till_metadata_id = m_base.id
    LEFT JOIN metadata_types mt_related ON m_related.metadatatyp_id = mt_related.id
    WHERE
      m_base.objekt_id = ${objektId}
      AND mt_base.namn = ${baseMetadataTypNamn}
  `;

  const result = await db.execute(query);
  return result.rows as any[];
}

// ==========================================================================
// 7. GEOGRAFISK UPPLÖSNINGSORDNING
// ==========================================================================

export async function getGeographicPosition(objektId: number): Promise<GeographicPosi-
tion | null> {
  const objectWithMetadata = await getObjectWithAllMetadata(objektId);

  if (!objectWithMetadata) return null;

  // Prioritet 1: GPS (exakt)
  const gpsMetadata = objectWithMetadata.metadata.find(m => m.typ.namn === 'GPS');
  if (gpsMetadata && gpsMetadata.vardeString) {
    return {
      typ: 'GPS',
      precision: 'exakt',
      varde: gpsMetadata.vardeString,
      fromObject: gpsMetadata.fromObject,
    };
  }

  // Prioritet 2: What3words (medel)
  const w3wMetadata = objectWithMetadata.metadata.find(m => m.typ.namn === 'What3-
words');
  if (w3wMetadata && w3wMetadata.vardeString) {
    return {
      typ: 'What3words',
      precision: 'medel',
      varde: w3wMetadata.vardeString,
      fromObject: w3wMetadata.fromObject,
    };
  }

  // Prioritet 3: Adress (grov)
  const adressMetadata = objectWithMetadata.metadata.find(m => m.typ.namn ===
'Adress');
  if (adressMetadata && adressMetadata.vardeString) {
    return {
      typ: 'Adress',
```

```
      precision: 'grov',
      varde: adressMetadata.vardeString,
      fromObject: adressMetadata.fromObject,
    };
  }

  return null;
}

// ==============================================================================
// 8. HÄMTA KLUSTERTRÄD
// ==============================================================================

export interface ClusterTreeNode {
  id: number;
  namn: string;
  typ: 'organisatorisk' | 'fysisk';
  parentId: number | null;
  children: ClusterTreeNode[];
  level: number;
}

export async function getClusterTree(rootId: number): Promise<ClusterTreeNode | null>
{
  // Rekursiv CTE för att hämta hela trädet
  const treeQuery = sql`
    WITH RECURSIVE tree AS (
      -- Starta med root
      SELECT
        id,
        namn,
        typ,
        parent_id,
        0 as level,
        ARRAY[id] as path
      FROM cluster_objects
      WHERE id = ${rootId}

      UNION ALL

      -- Gå nedåt i hierarkin
      SELECT
        o.id,
        o.namn,
        o.typ,
        o.parent_id,
        t.level + 1,
        t.path || o.id
      FROM cluster_objects o
      INNER JOIN tree t ON o.parent_id = t.id
    )
    SELECT * FROM tree
    ORDER BY path
  `;

  const result = await db.execute(treeQuery);
  const rows = result.rows as any[];

  if (rows.length === 0) return null;

  // Bygg träd-struktur
  const nodeMap = new Map<number, ClusterTreeNode>();
```

```
  rows.forEach(row => {
    nodeMap.set(row.id, {
      id: row.id,
      namn: row.namn,
      typ: row.typ,
      parentId: row.parent_id,
      children: [],
      level: row.level,
    });
  });

  // Länka barn till föräldrar
  rows.forEach(row => {
    if (row.parent_id !== null) {
      const parent = nodeMap.get(row.parent_id);
      const child = nodeMap.get(row.id);
      if (parent && child) {
        parent.children.push(child);
      }
    }
  });

  return nodeMap.get(rootId) || null;
}

// ==============================================================================
// 9. HITTA OBJEKT MED SPECIFIK METADATA
// ==============================================================================

export async function findObjectsWithMetadata(
  metadataTypNamn: string,
  varde?: any
): Promise<ObjectWithMetadata[]> {
  let whereConditions = [eq(metadataTypes.namn, metadataTypNamn)];

  // Om värde specificerat, filtrera på det också
  if (varde !== undefined) {
    // Detta är förenklat - i produktion behöver man hantera olika datatyper
    whereConditions.push(eq(metadata.vardeString, String(varde)));
  }

  const results = await db
    .select({
      objekt: clusterObjects,
      metadata: metadata,
      metadataTyp: metadataTypes,
    })
    .from(metadata)
    .innerJoin(clusterObjects, eq(metadata.objektId, clusterObjects.id))
    .innerJoin(metadataTypes, eq(metadata.metadatatypId, metadataTypes.id))
    .where(and(...whereConditions));

  // Gruppera per objekt
  const objektMap = new Map<number, ObjectWithMetadata>();

  results.forEach(row => {
    if (!objektMap.has(row.objekt.id)) {
      objektMap.set(row.objekt.id, {
        ...row.objekt,
        metadata: [],
      });
    }
```

```
    const obj = objektMap.get(row.objekt.id)!;
    obj.metadata.push({
      ...row.metadata,
      typ: row.metadataTyp,
      source: 'local',
    });
  });

  return Array.from(objektMap.values());
}
```

Del 3: Backend API

3.1 API-endpoints

Express Setup: `src/server.ts`

```
import express from 'express';
import cors from 'cors';
import { objectRouter } from './routes/objects';
import { metadataRouter } from './routes/metadata';

const app = express();
const port = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/objects', objectRouter);
app.use('/api/metadata', metadataRouter);

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

// Error handling
app.use((err: any, req: express.Request, res: express.Response, next: ex-
press.NextFunction) => {
  console.error('Error:', err);
  res.status(500).json({
    error: 'Internal server error',
    message: err.message,
  });
});

app.listen(port, () => {
  console.log(`🚀 Server running on port ${port}`);
});
```

Objects Router: `src/routes/objects.ts`

```
import { Router } from 'express';
import {
  getObjectWithAllMetadata,
  getClusterTree,
  getGeographicPosition
} from '../db/queries';
import { db } from '../db';
import { clusterObjects } from '../db/schema';
import { eq } from 'drizzle-orm';

export const objectRouter = Router();

// ==============================================================================
// GET /api/objects/:id - Hämta objekt med all metadata (inkl. ärvd)
// ==============================================================================
objectRouter.get('/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    const objekt = await getObjectWithAllMetadata(id);

    if (!objekt) {
      return res.status(404).json({ error: 'Object not found' });
    }

    res.json(objekt);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// GET /api/objects/:id/metadata - Hämta ENDAST metadata för objekt
// ==============================================================================
objectRouter.get('/:id/metadata', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    const objekt = await getObjectWithAllMetadata(id);

    if (!objekt) {
      return res.status(404).json({ error: 'Object not found' });
    }

    // Returnera endast metadata
    res.json(objekt.metadata);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// GET /api/objects/:id/tree - Hämta klusterträd från objekt och nedåt
// ==============================================================================
```

```
objectRouter.get('/:id/tree', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    const tree = await getClusterTree(id);

    if (!tree) {
      return res.status(404).json({ error: 'Object not found' });
    }

    res.json(tree);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// GET /api/objects/:id/position - Hämta geografisk position (med prioritering)
// ==============================================================================
objectRouter.get('/:id/position', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    const position = await getGeographicPosition(id);

    if (!position) {
      return res.status(404).json({ error: 'No geographic data found for object' });
    }

    res.json(position);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// POST /api/objects - Skapa nytt objekt
// ==============================================================================
objectRouter.post('/', async (req, res, next) => {
  try {
    const { namn, typ, parentId, skapadAv } = req.body;

    // Validering
    if (!namn || !typ) {
      return res.status(400).json({
        error: 'Missing required fields: namn, typ'
      });
    }

    if (typ !== 'organisatorisk' && typ !== 'fysisk') {
      return res.status(400).json({
        error: 'typ must be "organisatorisk" or "fysisk"'
      });
    }
```

```
    // Skapa objekt
    const [newObjekt] = await db.insert(clusterObjects).values({
      namn,
      typ,
      parentId: parentId || null,
      skapadAv,
    }).returning();

    res.status(201).json(newObjekt);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// PUT /api/objects/:id - Uppdatera objekt
// ==============================================================================
objectRouter.put('/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);
    const { namn, typ, parentId, uppdateradAv } = req.body;

    if (isNaN(id)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    // Kontrollera att objektet finns
    const [existing] = await db
      .select()
      .from(clusterObjects)
      .where(eq(clusterObjects.id, id));

    if (!existing) {
      return res.status(404).json({ error: 'Object not found' });
    }

    // Uppdatera
    const [updated] = await db
      .update(clusterObjects)
      .set({
        namn: namn || existing.namn,
        typ: typ || existing.typ,
        parentId: parentId !== undefined ? parentId : existing.parentId,
        uppdateradAv,
        uppdateradDatum: new Date(),
      })
      .where(eq(clusterObjects.id, id))
      .returning();

    res.json(updated);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// DELETE /api/objects/:id - Radera objekt
// ==============================================================================
objectRouter.delete('/:id', async (req, res, next) => {
  try {
    const id = parseInt(req.params.id);

    if (isNaN(id)) {
```

```
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    // Radera (CASCADE tar hand om metadata)
    await db.delete(clusterObjects).where(eq(clusterObjects.id, id));

    res.status(204).send();
  } catch (error) {
    next(error);
  }
});
```

Metadata Router: `src/routes/metadata.ts`

```
import { Router } from 'express';
import {
  createMetadata,
  updateMetadata,
  deleteMetadata,
  getCrossFertilizedMetadata,
  getMetadataValue
} from '../db/queries';

export const metadataRouter = Router();

// ==============================================================================
// POST /api/objects/:id/metadata - Skapa metadata för objekt
// ==============================================================================
metadataRouter.post('/objects/:id/metadata', async (req, res, next) => {
  try {
    const objektId = parseInt(req.params.id);
    const { metadataTypNamn, varde, arvsNedat, koppladTillMetadataId, skapadAv } = req
.body;

    if (isNaN(objektId)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    if (!metadataTypNamn || varde === undefined) {
      return res.status(400).json({
        error: 'Missing required fields: metadataTypNamn, varde'
      });
    }

    const newMetadata = await createMetadata({
      objektId,
      metadataTypNamn,
      varde,
      arvsNedat,
      koppladTillMetadataId,
      skapadAv,
    });

    res.status(201).json(newMetadata);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// PUT /api/metadata/:id - Uppdatera metadata
// ==============================================================================
metadataRouter.put('/:id', async (req, res, next) => {
  try {
    const metadataId = parseInt(req.params.id);
    const { varde, uppdateradAv } = req.body;

    if (isNaN(metadataId)) {
      return res.status(400).json({ error: 'Invalid metadata ID' });
    }

    if (varde === undefined) {
      return res.status(400).json({ error: 'Missing required field: varde' });
    }

    const updated = await updateMetadata(metadataId, varde, uppdateradAv);
```

```
    res.json(updated);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// DELETE /api/metadata/:id - Radera metadata
// ==============================================================================
metadataRouter.delete('/:id', async (req, res, next) => {
  try {
    const metadataId = parseInt(req.params.id);

    if (isNaN(metadataId)) {
      return res.status(400).json({ error: 'Invalid metadata ID' });
    }

    await deleteMetadata(metadataId);

    res.status(204).send();
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// GET /api/objects/:id/metadata/:typ/crossfertilized - Hämta korsbefruktad metadata
// ==============================================================================
metadataRouter.get('/objects/:id/metadata/:typ/crossfertilized', async (req, res, next
) => {
  try {
    const objektId = parseInt(req.params.id);
    const metadataTypNamn = req.params.typ;

    if (isNaN(objektId)) {
      return res.status(400).json({ error: 'Invalid object ID' });
    }

    const crossFertilized = await getCrossFertilizedMetadata(objektId, metadataTyp-
Namn);

    res.json(crossFertilized);
  } catch (error) {
    next(error);
  }
});

// ==============================================================================
// POST /api/metadata/crossfertilize - Koppla metadata till metadata
// ==============================================================================
metadataRouter.post('/crossfertilize', async (req, res, next) => {
  try {
    const {
      objektId,
      baseMetadataId,
      newMetadataTypNamn,
      newMetadataVarde,
      skapadAv
    } = req.body;

    if (!objektId || !baseMetadataId || !newMetadataTypNamn || newMetadataVarde === un
defined) {
```

```
      return res.status(400).json({
        error:
'Missing required fields: objektId, baseMetadataId, newMetadataTypNamn, newMetadataV-
arde'
      });
    }

    // Skapa ny metadata som är kopplad till bas-metadatan
    const newMetadata = await createMetadata({
      objektId,
      metadataTypNamn: newMetadataTypNamn,
      varde: newMetadataVarde,
      koppladTillMetadataId: baseMetadataId,
      skapadAv,
    });

    res.status(201).json(newMetadata);
  } catch (error) {
    next(error);
  }
});
```

3.2 Service Layer

För att hålla business logic separerad från routes, skapa service-lager.

Metadata Service: `src/services/MetadataService.ts`

```
import {
  getObjectWithAllMetadata,
  createMetadata as dbCreateMetadata,
  updateMetadata as dbUpdateMetadata,
  deleteMetadata as dbDeleteMetadata,
  getCrossFertilizedMetadata,
  getMetadataValue,
  type ObjectWithMetadata,
  type MetadataWithType,
} from '../db/queries';

export class MetadataService {
  /**
   * Hämta all metadata för ett objekt (inkl. ärvd)
   */
  async getAllMetadata(objektId: number): Promise<MetadataWithType[]> {
    const objekt = await getObjectWithAllMetadata(objektId);
    if (!objekt) {
      throw new Error(`Object with id ${objektId} not found`);
    }
    return objekt.metadata;
  }

  /**
   * Hämta endast lokala metadata (inte ärvda)
   */
  async getLocalMetadata(objektId: number): Promise<MetadataWithType[]> {
    const allMetadata = await this.getAllMetadata(objektId);
    return allMetadata.filter(m => m.source === 'local');
  }

  /**
   * Hämta endast ärvda metadata
   */
  async getInheritedMetadata(objektId: number): Promise<MetadataWithType[]> {
    const allMetadata = await this.getAllMetadata(objektId);
    return allMetadata.filter(m => m.source === 'inherited');
  }

  /**
   * Hämta ett specifikt metadata-värde
   */
  async getValue(objektId: number, metadataTypNamn: string): Promise<any> {
    return await getMetadataValue(objektId, metadataTypNamn);
  }

  /**
   * Skapa ny metadata
   */
  async create(data: {
    objektId: number;
    metadataTypNamn: string;
    varde: any;
    arvsNedat?: boolean;
    koppladTillMetadataId?: number;
    skapadAv?: string;
  }) {
    return await dbCreateMetadata(data);
  }

  /**
   * Uppdatera metadata
```

```
   */
  async update(metadataId: number, varde: any, uppdateradAv?: string) {
    return await dbUpdateMetadata(metadataId, varde, uppdateradAv);
  }

  /**
   * Radera metadata
   */
  async delete(metadataId: number) {
    await dbDeleteMetadata(metadataId);
  }

  /**
   * Koppla metadata till annan metadata (korsbefruktning)
   */
  async crossFertilize(data: {
    objektId: number;
    baseMetadataId: number;
    newMetadataTypNamn: string;
    newMetadataVarde: any;
    skapadAv?: string;
  }) {
    return await dbCreateMetadata({
      objektId: data.objektId,
      metadataTypNamn: data.newMetadataTypNamn,
      varde: data.newMetadataVarde,
      koppladTillMetadataId: data.baseMetadataId,
      skapadAv: data.skapadAv,
    });
  }

  /**
   * Hämta all metadata som är kopplad till en bas-metadata
   */
  async getRelatedMetadata(objektId: number, baseMetadataTypNamn: string) {
    return await getCrossFertilizedMetadata(objektId, baseMetadataTypNamn);
  }

  /**
   * Validera att metadata-värde är giltigt för sin typ
   */
  validateValue(metadataTyp: string, varde: any): { valid: boolean; error?: string } {
    // Implementera validering baserat på metadatatyp
    // Detta är en förenklad version

    if (varde === null || varde === undefined) {
      return { valid: false, error: 'Value cannot be null or undefined' };
    }

    // Lägg till mer specifik validering här
    return { valid: true };
  }
}

export const metadataService = new MetadataService();
```

Cluster Service: `src/services/ClusterService.ts`

```
import { getClusterTree, type ClusterTreeNode } from '../db/queries';
import { db } from '../db';
import { clusterObjects } from '../db/schema';
import { eq, sql } from 'drizzle-orm';

export class ClusterService {
  /**
   * Hämta hela klusterträdet från ett objekt
   */
  async getTree(rootId: number): Promise<ClusterTreeNode | null> {
    return await getClusterTree(rootId);
  }

  /**
   * Hämta alla barn till ett objekt (bara en nivå)
   */
  async getChildren(parentId: number) {
    return await db
      .select()
      .from(clusterObjects)
      .where(eq(clusterObjects.parentId, parentId));
  }

  /**
   * Hämta förälder till ett objekt
   */
  async getParent(objektId: number) {
    const [objekt] = await db
      .select()
      .from(clusterObjects)
      .where(eq(clusterObjects.id, objektId));

    if (!objekt || !objekt.parentId) {
      return null;
    }

    const [parent] = await db
      .select()
      .from(clusterObjects)
      .where(eq(clusterObjects.id, objekt.parentId));

    return parent || null;
  }

  /**
   * Hämta alla föräldrar upp till root (parent chain)
   */
  async getParentChain(objektId: number) {
    const query = sql`
      WITH RECURSIVE parent_chain AS (
        SELECT
          id,
          parent_id,
          namn,
          typ,
          0 as level
        FROM cluster_objects
        WHERE id = ${objektId}

        UNION ALL

        SELECT
```

```
          o.id,
          o.parent_id,
          o.namn,
          o.typ,
          pc.level + 1
        FROM cluster_objects o
        INNER JOIN parent_chain pc ON o.id = pc.parent_id
      )
      SELECT * FROM parent_chain
      ORDER BY level
    `;

    const result = await db.execute(query);
    return result.rows as any[];
  }

  /**
   * Flytta objekt till ny förälder
   */
  async move(objektId: number, newParentId: number | null) {
    // Validera att vi inte skapar cirkulär referens
    if (newParentId !== null) {
      const wouldCreateCycle = await this.wouldCreateCycle(objektId, newParentId);
      if (wouldCreateCycle) {
        throw new Error('Cannot move object: would create circular reference');
      }
    }

    const [updated] = await db
      .update(clusterObjects)
      .set({
        parentId: newParentId,
        uppdateradDatum: new Date(),
      })
      .where(eq(clusterObjects.id, objektId))
      .returning();

    return updated;
  }

  /**
   * Kontrollera om en flytt skulle skapa cirkulär referens
   */
  private async wouldCreateCycle(objektId: number, newParentId: number): Promise<boole
an> {
    // Om vi försöker göra objektet till sin egen förälder
    if (objektId === newParentId) {
      return true;
    }

    // Hämta alla föräldrar till newParent
    const parentChain = await this.getParentChain(newParentId);

    // Om objektId finns i parent chain, skulle det skapa cykel
    return parentChain.some((p: any) => p.id === objektId);
  }

  /**
   * Räkna antal objekt i ett träd
   */
  async countObjectsInTree(rootId: number): Promise<number> {
    const tree = await this.getTree(rootId);
    if (!tree) return 0;
```

```
    const countNodes = (node: ClusterTreeNode): number => {
      return 1 + node.children.reduce((sum, child) => sum + countNodes(child), 0);
    };

    return countNodes(tree);
  }

  /**
   * Platta ut träd till lista
   */
  flattenTree(tree: ClusterTreeNode): ClusterTreeNode[] {
    const result: ClusterTreeNode[] = [tree];

    tree.children.forEach(child => {
      result.push(...this.flattenTree(child));
    });

    return result;
  }
}

export const clusterService = new ClusterService();
```

Geographic Service: `src/services/GeographicService.ts`

```
import { getGeographicPosition, type GeographicPosition } from '../db/queries';

export class GeographicService {
  /**
   * Hämta geografisk position med korrekt prioritering
   */
  async getPosition(objektId: number): Promise<GeographicPosition | null> {
    return await getGeographicPosition(objektId);
  }

  /**
   * Konvertera What3words till GPS-koordinater (kräver API-nyckel)
   */
  async what3wordsToGPS(what3words: string): Promise<{ lat: number; lng: number } | nu
ll> {
    // Detta kräver integration med What3words API
    // Placeholder implementation
    console.log(`Converting ${what3words} to GPS...`);
    return null;
  }

  /**
   * Konvertera adress till GPS-koordinater (geocoding)
   */
  async addressToGPS(address: string): Promise<{ lat: number; lng: number } | null> {
    // Detta kräver integration med Google Maps API eller liknande
    // Placeholder implementation
    console.log(`Geocoding ${address}...`);
    return null;
  }

  /**
   * Beräkna avstånd mellan två GPS-koordinater (Haversine formula)
   */
  calculateDistance(
    lat1: number,
    lng1: number,
    lat2: number,
    lng2: number
  ): number {
    const R = 6371; // Jordens radie i km
    const dLat = this.toRad(lat2 - lat1);
    const dLng = this.toRad(lng2 - lng1);

    const a =
      Math.sin(dLat / 2) * Math.sin(dLat / 2) +
      Math.cos(this.toRad(lat1)) *
        Math.cos(this.toRad(lat2)) *
        Math.sin(dLng / 2) *
        Math.sin(dLng / 2);

    const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    const distance = R * c;

    return distance; // km
  }

  private toRad(degrees: number): number {
    return degrees * (Math.PI / 180);
  }

  /**
```

```
   * Parse GPS-sträng till koordinater
   */
  parseGPS(gpsString: string): { lat: number; lng: number } | null {
    const parts = gpsString.split(',').map(p => parseFloat(p.trim()));

    if (parts.length !== 2 || parts.some(isNaN)) {
      return null;
    }

    return { lat: parts[0], lng: parts[1] };
  }
}

export const geographicService = new GeographicService();
```

Del 4: Frontend Implementation

4.1 Komponenter

API Client: `src/lib/api.ts`

```
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3000/api';

export interface ClusterObject {
  id: number;
  namn: string;
  typ: 'organisatorisk' | 'fysisk';
  parentId: number | null;
}

export interface MetadataType {
  id: number;
  namn: string;
  datatyp: string;
  arLogisk: boolean;
}

export interface Metadata {
  id: number;
  objektId: number;
  metadatatypId: number;
  vardeString?: string | null;
  vardeInteger?: number | null;
  vardeDecimal?: string | null;
  vardeBoolean?: boolean | null;
  vardeDatetime?: string | null;
  vardeJson?: any | null;
  vardeReferens?: string | null;
  arvsNedat: boolean;
  koppladTillMetadataId?: number | null;
  typ: MetadataType;
  source: 'local' | 'inherited';
  fromObject?: {
    id: number;
    namn: string;
    level: number;
  };
}

export interface ObjectWithMetadata extends ClusterObject {
  metadata: Metadata[];
}

export interface ClusterTreeNode extends ClusterObject {
  children: ClusterTreeNode[];
  level: number;
}

// ==============================================================================
// API FUNCTIONS
// ==============================================================================

export async function fetchObject(id: number): Promise<ObjectWithMetadata> {
  const response = await fetch(`${API_BASE_URL}/objects/${id}`);
  if (!response.ok) {
    throw new Error(`Failed to fetch object: ${response.statusText}`);
  }
  return response.json();
}

export async function fetchObjectMetadata(id: number): Promise<Metadata[]> {
  const response = await fetch(`${API_BASE_URL}/objects/${id}/metadata`);
  if (!response.ok) {
```

```
    throw new Error(`Failed to fetch metadata: ${response.statusText}`);
  }
  return response.json();
}

export async function fetchClusterTree(id: number): Promise<ClusterTreeNode> {
  const response = await fetch(`${API_BASE_URL}/objects/${id}/tree`);
  if (!response.ok) {
    throw new Error(`Failed to fetch cluster tree: ${response.statusText}`);
  }
  return response.json();
}

export async function createObject(data: {
  namn: string;
  typ: 'organisatorisk' | 'fysisk';
  parentId?: number | null;
}): Promise<ClusterObject> {
  const response = await fetch(`${API_BASE_URL}/objects`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    throw new Error(`Failed to create object: ${response.statusText}`);
  }
  return response.json();
}

export async function updateObject(
  id: number,
  data: Partial<ClusterObject>
): Promise<ClusterObject> {
  const response = await fetch(`${API_BASE_URL}/objects/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    throw new Error(`Failed to update object: ${response.statusText}`);
  }
  return response.json();
}

export async function deleteObject(id: number): Promise<void> {
  const response = await fetch(`${API_BASE_URL}/objects/${id}`, {
    method: 'DELETE',
  });
  if (!response.ok) {
    throw new Error(`Failed to delete object: ${response.statusText}`);
  }
}

export async function createMetadata(data: {
  objektId: number;
  metadataTypNamn: string;
  varde: any;
  arvsNedat?: boolean;
  koppladTillMetadataId?: number;
}): Promise<Metadata> {
  const response = await fetch(`${API_BASE_URL}/objects/${data.objektId}/metadata`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    throw new Error(`Failed to create metadata: ${response.statusText}`);
  }
  return response.json();
}

export async function updateMetadata(
  metadataId: number,
  varde: any
): Promise<Metadata> {
  const response = await fetch(`${API_BASE_URL}/metadata/${metadataId}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ varde }),
  });
  if (!response.ok) {
    throw new Error(`Failed to update metadata: ${response.statusText}`);
  }
  return response.json();
}

export async function deleteMetadata(metadataId: number): Promise<void> {
  const response = await fetch(`${API_BASE_URL}/metadata/${metadataId}`, {
    method: 'DELETE',
  });
  if (!response.ok) {
    throw new Error(`Failed to delete metadata: ${response.statusText}`);
  }
}

export async function crossFertilizeMetadata(data: {
  objektId: number;
  baseMetadataId: number;
  newMetadataTypNamn: string;
  newMetadataVarde: any;
}): Promise<Metadata> {
  const response = await fetch(`${API_BASE_URL}/metadata/crossfertilize`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    throw new Error(`Failed to cross-fertilize metadata: ${response.statusText}`);
  }
  return response.json();
}
```

ClusterTreeView Component: `src/components/ClusterTreeView.tsx`

```
'use client';

import React, { useState, useEffect } from 'react';
import { fetchClusterTree, type ClusterTreeNode } from '@/lib/api';

interface ClusterTreeViewProps {
  rootId: number;
  onSelectNode?: (node: ClusterTreeNode) => void;
}

export function ClusterTreeView({ rootId, onSelectNode }: ClusterTreeViewProps) {
  const [tree, setTree] = useState<ClusterTreeNode | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [expandedNodes, setExpandedNodes] = useState<Set<number>>(new Set([rootId]));
  const [selectedNodeId, setSelectedNodeId] = useState<number | null>(null);

  useEffect(() => {
    loadTree();
  }, [rootId]);

  async function loadTree() {
    setLoading(true);
    setError(null);
    try {
      const data = await fetchClusterTree(rootId);
      setTree(data);
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  function toggleNode(nodeId: number) {
    setExpandedNodes(prev => {
      const next = new Set(prev);
      if (next.has(nodeId)) {
        next.delete(nodeId);
      } else {
        next.add(nodeId);
      }
      return next;
    });
  }

  function handleSelectNode(node: ClusterTreeNode) {
    setSelectedNodeId(node.id);
    onSelectNode?.(node);
  }

  function renderNode(node: ClusterTreeNode) {
    const isExpanded = expandedNodes.has(node.id);
    const isSelected = selectedNodeId === node.id;
    const hasChildren = node.children.length > 0;

    return (
      <div key={node.id} style={{ marginLeft: `${node.level * 20}px` }}>
        <div
          style={{
            padding: '8px',
            cursor: 'pointer',
```

```
            backgroundColor: isSelected ? '#e3f2fd' : 'transparent',
            borderRadius: '4px',
            display: 'flex',
            alignItems: 'center',
            gap: '8px',
          }}
          onClick={() => handleSelectNode(node)}
        >
          {hasChildren && (
            <button
              onClick={(e) => {
                e.stopPropagation();
                toggleNode(node.id);
              }}
              style={{
                border: 'none',
                background: 'none',
                cursor: 'pointer',
                fontSize: '16px',
              }}
            >
              {isExpanded ? '▼' : '▶'}
            </button>
          )}

          <span style={{ marginLeft: hasChildren ? 0 : '24px' }}>
            {node.typ === 'fysisk' ? '🟢' : '🔵'} {node.namn}
          </span>

          <span style={{ fontSize: '12px', color: '#666' }}>
            ({node.typ})
          </span>
        </div>

        {isexpanded && hasChildren && (
          <div>
            {node.children.map(child => renderNode(child))}
          </div>
        )}
      </div>
    );
  }

  if (loading) {
    return <div>Laddar klusterträd...</div>;
  }

  if (error) {
    return <div style={{ color: 'red' }}>Fel: {error}</div>;
  }

  if (!tree) {
    return <div>Inget träd att visa</div>;
  }

  return (
    <div style={{ border: '1px solid #ddd', borderRadius: '4px', padding: '16px' }}>
      <h3>Klusterträd</h3>
      {renderNode(tree)}
    </div>
  );
}
```

MetadataList Component: `src/components/MetadataList.tsx`

```
'use client';

import React, { useState, useEffect } from 'react';
import { fetchObjectMetadata, type Metadata } from '@/lib/api';

interface MetadataListProps {
  objektId: number;
  onEdit?: (metadata: Metadata) => void;
  onDelete?: (metadataId: number) => void;
}

export function MetadataList({ objektId, onEdit, onDelete }: MetadataListProps) {
  const [metadata, setMetadata] = useState<Metadata[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [filter, setFilter] = useState<'all' | 'local' | 'inherited'>('all');

  useEffect(() => {
    loadMetadata();
  }, [objektId]);

  async function loadMetadata() {
    setLoading(true);
    setError(null);
    try {
      const data = await fetchObjectMetadata(objektId);
      setMetadata(data);
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  function getMetadataValue(m: Metadata): string {
    if (m.vardeString !== null && m.vardeString !== undefined) return m.vardeString;
    if (m.vardeInteger !== null && m.vardeInteger !== undefined) return String(m.varde
Integer);
    if (m.vardeDecimal !== null && m.vardeDecimal !== undefined) return m.vardeDecimal
;
    if (m.vardeBoolean !== null && m.vardeBoolean !== undefined) return
m.vardeBoolean ? 'Ja' : 'Nej';
    if (m.vardeDatetime !== null && m.vardeDatetime !== undefined) return new Date(m.v
ardeDatetime).toLocaleString('sv-SE');
    if (m.vardeJson !== null && m.vardeJson !== undefined) return JSON.stringi-
fy(m.vardeJson);
    if (m.vardeReferens !== null && m.vardeReferens !== undefined) return m.vardeRefer
ens;
    return '-';
  }

  const filteredMetadata = metadata.filter(m => {
    if (filter === 'all') return true;
    return m.source === filter;
  });

  const localMetadata = metadata.filter(m => m.source === 'local');
  const inheritedMetadata = metadata.filter(m => m.source === 'inherited');

  if (loading) {
    return <div>Laddar metadata...</div>;
  }
```

```
  if (error) {
    return <div style={{ color: 'red' }}>Fel: {error}</div>;
  }

  return (
    <div style={{ border: '1px solid #ddd', borderRadius: '4px', padding: '16px' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'cen
ter', marginBottom: '16px' }}>
        <h3>Metadata</h3>
        <div style={{ display: 'flex', gap: '8px' }}>
          <button
            onClick={() => setFilter('all')}
            style={{
              padding: '4px 12px',
              border: '1px solid #ddd',
              borderRadius: '4px',
              backgroundColor: filter === 'all' ? '#2196f3' : 'white',
              color: filter === 'all' ? 'white' : 'black',
              cursor: 'pointer',
            }}
          >
            Alla ({metadata.length})
          </button>
          <button
            onClick={() => setFilter('local')}
            style={{
              padding: '4px 12px',
              border: '1px solid #ddd',
              borderRadius: '4px',
              backgroundColor: filter === 'local' ? '#4caf50' : 'white',
              color: filter === 'local' ? 'white' : 'black',
              cursor: 'pointer',
            }}
          >
            Lokala ({localMetadata.length})
          </button>
          <button
            onClick={() => setFilter('inherited')}
            style={{
              padding: '4px 12px',
              border: '1px solid #ddd',
              borderRadius: '4px',
              backgroundColor: filter === 'inherited' ? '#ff9800' : 'white',
              color: filter === 'inherited' ? 'white' : 'black',
              cursor: 'pointer',
            }}
          >
            Ärvda ({inheritedMetadata.length})
          </button>
        </div>
      </div>

      {filteredMetadata.length === 0 ? (
        <div style={{ color: '#666', fontStyle: 'italic' }}>Ingen metadata att visa</
div>
      ) : (
        <table style={{ width: '100%', borderCollapse: 'collapse' }}>
          <thead>
            <tr style={{ borderBottom: '2px solid #ddd' }}>
              <th style={{ padding: '8px', textAlign: 'left' }}>Typ</th>
              <th style={{ padding: '8px', textAlign: 'left' }}>Värde</th>
              <th style={{ padding: '8px', textAlign: 'left' }}>Källa</th>
```

```
              <th style={{ padding: '8px', textAlign: 'left' }}>Ärvs</th>
              <th style={{ padding: '8px', textAlign: 'right' }}>Åtgärder</th>
            </tr>
          </thead>
          <tbody>
            {filteredMetadata.map(m => (
              <tr key={m.id} style={{ borderBottom: '1px solid #eee' }}>
                <td style={{ padding: '8px' }}>
                  <strong>{m.typ.namn}</strong>
                  {m.typ.arLogisk && <span style={{ marginLeft: '4px', fontSize: '12px
', color: '#4caf50' }}>⚡</span>}
                </td>
                <td style={{ padding: '8px' }}>{getMetadataValue(m)}</td>
                <td style={{ padding: '8px' }}>
                  {m.source === 'local' ? (
                    <span style={{ color: '#4caf50', fontWeight: 'bold' }}>Lokalt</
span>
                  ) : (
                    <span style={{ color: '#ff9800' }}>
                      Ärvt från <strong>{m.fromObject?.namn}</strong>
                    </span>
                  )}
                </td>
                <td style={{ padding: '8px' }}>
                  {m.arvsNedat ? '✓' : '-'}
                </td>
                <td style={{ padding: '8px', textAlign: 'right' }}>
                  {m.source === 'local' && (
                    <div style={{ display: 'flex', gap: '4px', justifyContent: 'flex-
end' }}>
                      <button
                        onClick={() => onEdit?.(m)}
                        style={{
                          padding: '4px 8px',
                          border: '1px solid #2196f3',
                          borderRadius: '4px',
                          backgroundColor: 'white',
                          color: '#2196f3',
                          cursor: 'pointer',
                        }}
                      >
                        Redigera
                      </button>
                      <button
                        onClick={() => onDelete?.(m.id)}
                        style={{
                          padding: '4px 8px',
                          border: '1px solid #f44336',
                          borderRadius: '4px',
                          backgroundColor: 'white',
                          color: '#f44336',
                          cursor: 'pointer',
                        }}
                      >
                        Ta bort
                      </button>
                    </div>
                  )}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
```

```
      )}
    </div>
  );
}
```

MetadataEditor Component: `src/components/MetadataEditor.tsx`

```
'use client';

import React, { useState } from 'react';
import { createMetadata, updateMetadata, type Metadata } from '@/lib/api';

interface MetadataEditorProps {
  objektId: number;
  existingMetadata?: Metadata | null;
  onSave?: () => void;
  onCancel?: () => void;
}

const METADATA_TYPES = [
  'Adress',
  'GPS',
  'What3words',
  'Antal',
  'Area',
  'Volym',
  'Detaljtyp',
  'Beskrivning',
  'Anteckningar',
  'Kod',
  'Frekvens',
];

export function MetadataEditor({ objektId, existingMetadata, onSave, onCancel }: Metad
ataEditorProps) {
  const [metadataTypNamn, setMetadataTypNamn] = useState(existingMetadata?.typ.namn
|| '');
  const [varde, setVarde] = useState(
    existingMetadata
      ? (existingMetadata.vardeString ||
         existingMetadata.vardeInteger?.toString() ||
         existingMetadata.vardeDecimal ||
         '')
      : ''
  );
  const [arvsNedat, setArvsNedat] = useState(existingMetadata?.arvsNedat || false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();

    if (!metadataTypNamn || !varde) {
      setError('Både typ och värde måste anges');
      return;
    }

    setLoading(true);
    setError(null);

    try {
      if (existingMetadata) {
        // Uppdatera befintlig metadata
        await updateMetadata(existingMetadata.id, varde);
      } else {
        // Skapa ny metadata
        await createMetadata({
          objektId,
          metadataTypNamn,
```

```
          varde,
          arvsNedat,
        });
      }

      onSave?.();
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  return (
    <div style={{ border: '1px solid #ddd', borderRadius: '4px', padding: '16px' }}>
      <h3>{existingMetadata ? 'Redigera Metadata' : 'Lägg till Metadata'}</h3>

      {error && (
        <div style={{ padding: '8px', backgroundColor: '#ffebee', color: '#c62828', bo
rderRadius: '4px', marginBottom: '16px' }}>
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit}>
        <div style={{ marginBottom: '16px' }}>
          <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}
>
            Metadatatyp
          </label>
          <select
            value={metadataTypNamn}
            onChange={(e) => setMetadataTypNamn(e.target.value)}
            disabled={!!existingMetadata} // Kan inte ändra typ på befintlig metadata
            style={{
              width: '100%',
              padding: '8px',
              border: '1px solid #ddd',
              borderRadius: '4px',
            }}
            required
          >
            <option value="">Välj typ...</option>
            {METADATA_TYPES.map(typ => (
              <option key={typ} value={typ}>{typ}</option>
            ))}
          </select>
        </div>

        <div style={{ marginBottom: '16px' }}>
          <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}
>
            Värde
          </label>
          <input
            type="text"
            value={varde}
            onChange={(e) => setVarde(e.target.value)}
            style={{
              width: '100%',
              padding: '8px',
              border: '1px solid #ddd',
              borderRadius: '4px',
```

```
            }}
            required
          />
        </div>

        {!existingMetadata && (
          <div style={{ marginBottom: '16px' }}>
            <label style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
              <input
                type="checkbox"
                checked={arvsNedat}
                onChange={(e) => setArvsNedat(e.target.checked)}
              />
              <span>Ärvs nedåt till barn-objekt</span>
            </label>
          </div>
        )}

        <div style={{ display: 'flex', gap: '8px', justifyContent: 'flex-end' }}>
          {onCancel && (
            <button
              type="button"
              onClick={onCancel}
              style={{
                padding: '8px 16px',
                border: '1px solid #ddd',
                borderRadius: '4px',
                backgroundColor: 'white',
                cursor: 'pointer',
              }}
            >
              Avbryt
            </button>
          )}
          <button
            type="submit"
            disabled={loading}
            style={{
              padding: '8px 16px',
              border: 'none',
              borderRadius: '4px',
              backgroundColor: '#2196f3',
              color: 'white',
              cursor: loading ? 'not-allowed' : 'pointer',
            }}
          >
            {loading ? 'Sparar...' : 'Spara'}
          </button>
        </div>
      </form>
    </div>
  );
}
```

Main Object View: `src/components/ObjectView.tsx`

```
'use client';

import React, { useState, useEffect } from 'react';
import { ClusterTreeView } from './ClusterTreeView';
import { MetadataList } from './MetadataList';
import { MetadataEditor } from './MetadataEditor';
import { fetchObject, deleteMetadata, type ObjectWithMetadata, type Metadata } from '@
/lib/api';

interface ObjectViewProps {
  objektId: number;
}

export function ObjectView({ objektId }: ObjectViewProps) {
  const [objekt, setObjekt] = useState<ObjectWithMetadata | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [showEditor, setShowEditor] = useState(false);
  const [editingMetadata, setEditingMetadata] = useState<Metadata | null>(null);
  const [refreshKey, setRefreshKey] = useState(0);

  useEffect(() => {
    loadObject();
  }, [objektId, refreshKey]);

  async function loadObject() {
    setLoading(true);
    setError(null);
    try {
      const data = await fetchObject(objektId);
      setObjekt(data);
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  async function handleDeleteMetadata(metadataId: number) {
    if (!confirm('Är du säker på att du vill ta bort denna metadata?')) {
      return;
    }

    try {
      await deleteMetadata(metadataId);
      setRefreshKey(prev => prev + 1);
    } catch (err: any) {
      alert(`Fel vid borttagning: ${err.message}`);
    }
  }

  function handleEditMetadata(metadata: Metadata) {
    setEditingMetadata(metadata);
    setShowEditor(true);
  }

  function handleSaveMetadata() {
    setShowEditor(false);
    setEditingMetadata(null);
    setRefreshKey(prev => prev + 1);
  }
```

```
function handleCancelEdit() {
  setShowEditor(false);
  setEditingMetadata(null);
}

if (loading) {
  return <div>Laddar objekt...</div>;
}

if (error) {
  return <div style={{ color: 'red' }}>Fel: {error}</div>;
}

if (!objekt) {
  return <div>Inget objekt att visa</div>;
}

return (
  <div style={{ padding: '24px' }}>
    <div style={{ marginBottom: '24px' }}>
      <h1>{objekt.namn}</h1>
      <div style={{ display: 'flex', gap: '16px', color: '#666' }}>
        <span>ID: {objekt.id}</span>
        <span>Typ: {objekt.typ}</span>
        {objekt.parentId && <span>Förälder-ID: {objekt.parentId}</span>}
      </div>
    </div>

    <div style={{ display: 'grid', gridTemplateColumns: '1fr 2fr', gap: '24px' }}>
      <div>
        <ClusterTreeView rootId={objekt.parentId || objekt.id} />
      </div>

      <div>
        <div style={{ marginBottom: '16px' }}>
          {!showEditor && (
            <button
              onClick={() => setShowEditor(true)}
              style={{
                padding: '8px 16px',
                border: 'none',
                borderRadius: '4px',
                backgroundColor: '#4caf50',
                color: 'white',
                cursor: 'pointer',
              }}
            >
              + Lägg till Metadata
            </button>
          )}
        </div>

        {showEditor && (
          <div style={{ marginBottom: '24px' }}>
            <MetadataEditor
              objektId={objekt.id}
              existingMetadata={editingMetadata}
              onSave={handleSaveMetadata}
              onCancel={handleCancelEdit}
            />
          </div>
        )}
```

```
            <MetadataList
              objektId={objekt.id}
              onEdit={handleEditMetadata}
              onDelete={handleDeleteMetadata}
            />
          </div>
        </div>
      </div>
    );
  }
```

4.2 State Management

För enklare state management utan externa bibliotek, använd React Context.

Metadata Context: `src/contexts/MetadataContext.tsx`

```
'use client';

import React, { createContext, useContext, useState, useCallback } from 'react';
import type { Metadata, ObjectWithMetadata } from '@/lib/api';

interface MetadataContextType {
  cache: Map<number, ObjectWithMetadata>;
  getObject: (id: number) => ObjectWithMetadata | null;
  setObject: (id: number, obj: ObjectWithMetadata) => void;
  invalidate: (id: number) => void;
  invalidateAll: () => void;
}

const MetadataContext = createContext<MetadataContextType | null>(null);

export function MetadataProvider({ children }: { children: React.ReactNode }) {
  const [cache, setCache] = useState<Map<number, ObjectWithMetadata>>(new Map());

  const getObject = useCallback((id: number) => {
    return cache.get(id) || null;
  }, [cache]);

  const setObject = useCallback((id: number, obj: ObjectWithMetadata) => {
    setCache(prev => new Map(prev).set(id, obj));
  }, []);

  const invalidate = useCallback((id: number) => {
    setCache(prev => {
      const next = new Map(prev);
      next.delete(id);
      return next;
    });
  }, []);

  const invalidateAll = useCallback(() => {
    setCache(new Map());
  }, []);

  return (
    <MetadataContext.Provider value={{ cache, getObject, setObject, invalidate, inval-
idateAll }}>
      {children}
    </MetadataContext.Provider>
  );
}

export function useMetadataContext() {
  const context = useContext(MetadataContext);
  if (!context) {
    throw new Error('useMetadataContext must be used within MetadataProvider');
  }
  return context;
}
```

Del 5: Konkreta Implementationssteg

Steg 1: Setup Projekt

```
# Skapa projekt
mkdir kinab-objektdata-metadata
cd kinab-objektdata-metadata

# Backend
mkdir backend
cd backend
npm init -y
npm install express cors drizzle-orm postgres dotenv
npm install -D typescript @types/express @types/node @types/cors tsx drizzle-kit
npx tsc --init

# Frontend
cd ..
npx create-next-app@latest frontend --typescript --tailwind --app
cd frontend
npm install
```

Steg 2: Konfigurera Databas

```
# .env fil
echo "DATABASE_URL=postgresql://user:password@localhost:5432/kinab" > backend/.env

# Skapa databas
createdb kinab  # eller via PostgreSQL GUI
```

Steg 3: Implementera Schema

```
cd backend
mkdir -p src/db
# Kopiera schema.ts från Del 2.1
# Kopiera drizzle.config.ts från Del 2.1
```

Steg 4: Kör Migrationer

```
cd backend
npx drizzle-kit generate:pg
npx drizzle-kit push:pg
```

Steg 5: Seed Data

```
cd backend
# Kopiera seed.ts från Del 2.2
npm run db:seed
```

Steg 6: Implementera Queries

```
cd backend/src/db
# Kopiera queries.ts från Del 2.3
```

Steg 7: Implementera Services

```
cd backend
mkdir -p src/services
# Kopiera MetadataService.ts, ClusterService.ts, GeographicService.ts från Del 3.2
```

Steg 8: Implementera API Routes

```
cd backend
mkdir -p src/routes
# Kopiera objects.ts och metadata.ts från Del 3.1
# Kopiera server.ts från Del 3.1
```

Steg 9: Testa Backend

```
cd backend
npm run dev

# I en annan terminal, testa API
curl http://localhost:3000/health
curl http://localhost:3000/api/objects/1
curl http://localhost:3000/api/objects/1/metadata
```

Steg 10: Implementera Frontend

```
cd frontend
mkdir -p src/lib src/components src/contexts

# Kopiera api.ts från Del 4.1
# Kopiera alla komponenter från Del 4.1
# Kopiera MetadataContext.tsx från Del 4.2
```

Steg 11: Integrera Frontend med Backend

```
cd frontend
# Uppdatera .env.local
echo "NEXT_PUBLIC_API_URL=http://localhost:3000/api" > .env.local
```

Steg 12: Skapa Main Page

```
// frontend/src/app/page.tsx
'use client';

import { ObjectView } from '@/components/ObjectView';
import { MetadataProvider } from '@/contexts/MetadataContext';
import { useState } from 'react';

export default function Home() {
  const [objektId, setObjektId] = useState(1);

  return (
    <MetadataProvider>
      <div style={{ padding: '24px' }}>
        <div style={{ marginBottom: '24px' }}>
          <label>
            Objekt-ID:
            <input
              type="number"
              value={objektId}
              onChange={(e) => setObjektId(parseInt(e.target.value))}
              style={{ marginLeft: '8px', padding: '4px 8px' }}
            />
          </label>
        </div>

        <ObjectView objektId={objektId} />
      </div>
    </MetadataProvider>
  );
}
```

Steg 13: Kör Frontend

```
cd frontend
npm run dev
# Öppna http://localhost:3001
```

Steg 14: Verifiera med Test Cases

Se Del 6 för konkreta test cases.

Del 6: Testning

6.1 Unit Tests

Test Metadata-nedärvning: `backend/src/__tests__/metadata-inheritance.test.ts`

```
import { describe, test, expect, beforeAll, afterAll } from '@jest/globals';
import { db } from '../db';
import { clusterObjects, metadata, metadataTypes } from '../db/schema';
import { getObjectWithAllMetadata, createMetadata } from '../db/queries';

describe('Metadata Inheritance', () => {
  let parentObjektId: number;
  let childObjektId: number;
  let grandchildObjektId: number;

  beforeAll(async () => {
    // Skapa test-objekt
    const [parent] = await db.insert(clusterObjects).values({
      namn: 'Test Parent',
      typ: 'organisatorisk',
      parentId: null,
    }).returning();
    parentObjektId = parent.id;

    const [child] = await db.insert(clusterObjects).values({
      namn: 'Test Child',
      typ: 'organisatorisk',
      parentId: parentObjektId,
    }).returning();
    childObjektId = child.id;

    const [grandchild] = await db.insert(clusterObjects).values({
      namn: 'Test Grandchild',
      typ: 'fysisk',
      parentId: childObjektId,
    }).returning();
    grandchildObjektId = grandchild.id;
  });

  afterAll(async () => {
    // Cleanup
    await db.delete(clusterObjects).where(eq(clusterObjects.id, grandchildObjektId));
    await db.delete(clusterObjects).where(eq(clusterObjects.id, childObjektId));
    await db.delete(clusterObjects).where(eq(clusterObjects.id, parentObjektId));
  });

  test('Metadata with arvs_nedat=true should be inherited by child', async () => {
    // Skapa metadata på parent med arvs_nedat=true
    await createMetadata({
      objektId: parentObjektId,
      metadataTypNamn: 'Adress',
      varde: 'Test Street 123',
      arvsNedat: true,
    });

    // Hämta child med metadata
    const childWithMetadata = await getObjectWithAllMetadata(childObjektId);

    // Verifiera att adress är ärvd
    const adressMetadata = childWithMetadata?.metadata.find(m => m.typ.namn === 'Ad-
ress');
    expect(adressMetadata).toBeDefined();
    expect(adressMetadata?.vardeString).toBe('Test Street 123');
    expect(adressMetadata?.source).toBe('inherited');
  });

  test('Metadata with arvs_nedat=false should NOT be inherited', async () => {
```

```
    // Skapa metadata på parent med arvs_nedat=false
    await createMetadata({
      objektId: parentObjektId,
      metadataTypNamn: 'Antal',
      varde: 10,
      arvsNedat: false,
    });

    // Hämta child med metadata
    const childWithMetadata = await getObjectWithAllMetadata(childObjektId);

    // Verifiera att antal INTE är ärvd
    const antalMetadata = childWithMetadata?.metadata.find(m => m.typ.namn ===
'Antal');
    expect(antalMetadata).toBeUndefined();
  });

  test('Local metadata should override inherited metadata', async () => {
    // Skapa metadata på parent
    await createMetadata({
      objektId: parentObjektId,
      metadataTypNamn: 'Beskrivning',
      varde: 'Parent description',
      arvsNedat: true,
    });

    // Skapa metadata på child (samma typ)
    await createMetadata({
      objektId: childObjektId,
      metadataTypNamn: 'Beskrivning',
      varde: 'Child description',
      arvsNedat: false,
    });

    // Hämta child med metadata
    const childWithMetadata = await getObjectWithAllMetadata(childObjektId);

    // Verifiera att lokalt värde används (inte ärvt)
    const beskrivningMetadata = childWithMetadata?.metadata.find(
      m => m.typ.namn === 'Beskrivning' && m.source === 'local'
    );
    expect(beskrivningMetadata?.vardeString).toBe('Child description');
  });

  test('Metadata should be inherited through multiple levels', async () => {
    // Skapa metadata på parent
    await createMetadata({
      objektId: parentObjektId,
      metadataTypNamn: 'Kod',
      varde: '1234',
      arvsNedat: true,
    });

    // Hämta grandchild med metadata
    const grandchildWithMetadata = await getObjectWithAllMetadata(grandchildObjektId);

    // Verifiera att kod är ärvd från grandparent
    const kodMetadata = grandchildWithMetadata?.metadata.find(m => m.typ.namn === 'Kod
');
    expect(kodMetadata).toBeDefined();
    expect(kodMetadata?.vardeString).toBe('1234');
    expect(kodMetadata?.source).toBe('inherited');
    expect(kodMetadata?.fromObject?.id).toBe(parentObjektId);
```

```
  });
});
```

Test Korsbefruktning: `backend/src/__tests__/crossfertilization.test.ts`

```
import { describe, test, expect, beforeAll, afterAll } from '@jest/globals';
import { db } from '../db';
import { clusterObjects } from '../db/schema';
import { createMetadata, getCrossFertilizedMetadata } from '../db/queries';
import { metadataService } from '../services/MetadataService';

describe('Metadata Cross-Fertilization', () => {
  let objektId: number;

  beforeAll(async () => {
    const [objekt] = await db.insert(clusterObjects).values({
      namn: 'Test Object',
      typ: 'fysisk',
      parentId: null,
    }).returning();
    objektId = objekt.id;
  });

  afterAll(async () => {
    await db.delete(clusterObjects).where(eq(clusterObjects.id, objektId));
  });

  test('Should link Antal to Artikel metadata', async () => {
    // Skapa artikel-metadata
    const artikelMetadata = await createMetadata({
      objektId,
      metadataTypNamn: 'Artikel',
      varde: 'ART_TVATTA_PANTKARL',
      arvsNedat: false,
    });

    // Skapa antal-metadata kopplad till artikel
    await metadataService.crossFertilize({
      objektId,
      baseMetadataId: artikelMetadata.id,
      newMetadataTypNamn: 'Antal',
      newMetadataVarde: 10,
    });

    // Hämta korsbefruktad metadata
    const crossFertilized = await getCrossFertilizedMetadata(objektId, 'Artikel');

    // Verifiera
    expect(crossFertilized.length).toBeGreaterThan(0);
    const antalEntry = crossFertilized.find(cf => cf.related_typ === 'Antal');
    expect(antalEntry).toBeDefined();
    expect(antalEntry?.related_varde).toBe('10');
  });

  test('Should link Rating to Artikel metadata', async () => {
    // Skapa artikel-metadata
    const artikelMetadata = await createMetadata({
      objektId,
      metadataTypNamn: 'Artikel',
      varde: 'ART_RUMSTVATT',
      arvsNedat: false,
    });

    // Skapa rating-metadata kopplad till artikel
    await metadataService.crossFertilize({
      objektId,
      baseMetadataId: artikelMetadata.id,
```

```
      newMetadataTypNamn: 'Rating',
      newMetadataVarde: 4,
    });

    // Hämta korsbefruktad metadata
    const crossFertilized = await getCrossFertilizedMetadata(objektId, 'Artikel');

    // Verifiera
    const ratingEntry = crossFertilized.find(cf => cf.related_typ === 'Rating');
    expect(ratingEntry).toBeDefined();
    expect(ratingEntry?.related_varde).toBe('4');
  });
});
```

Test Geografisk Upplösningsordning: `backend/src/__tests__/geographic-resolution.test.ts`

```
import { describe, test, expect, beforeAll, afterAll } from '@jest/globals';
import { db } from '../db';
import { clusterObjects } from '../db/schema';
import { createMetadata } from '../db/queries';
import { geographicService } from '../services/GeographicService';

describe('Geographic Resolution Order', () => {
  let objektId: number;

  beforeAll(async () => {
    const [objekt] = await db.insert(clusterObjects).values({
      namn: 'Test Location Object',
      typ: 'fysisk',
      parentId: null,
    }).returning();
    objektId = objekt.id;
  });

  afterAll(async () => {
    await db.delete(clusterObjects).where(eq(clusterObjects.id, objektId));
  });

  test('Should prioritize GPS over What3words', async () => {
    // Skapa What3words metadata
    await createMetadata({
      objektId,
      metadataTypNamn: 'What3words',
      varde: 'test.words.location',
      arvsNedat: false,
    });

    // Skapa GPS metadata
    await createMetadata({
      objektId,
      metadataTypNamn: 'GPS',
      varde: '59.3156, 18.0649',
      arvsNedat: false,
    });

    // Hämta position
    const position = await geographicService.getPosition(objektId);

    // Verifiera att GPS används (högst prioritet)
    expect(position?.typ).toBe('GPS');
    expect(position?.precision).toBe('exakt');
    expect(position?.varde).toBe('59.3156, 18.0649');
  });

  test('Should prioritize What3words over Adress', async () => {
    // Rensa tidigare metadata
    await db.delete(metadata).where(eq(metadata.objektId, objektId));

    // Skapa Adress metadata
    await createMetadata({
      objektId,
      metadataTypNamn: 'Adress',
      varde: 'Test Street 123, Stockholm',
      arvsNedat: false,
    });

    // Skapa What3words metadata
    await createMetadata({
```

```
      objektId,
      metadataTypNamn: 'What3words',
      varde: 'test.words.location',
      arvsNedat: false,
    });

    // Hämta position
    const position = await geographicService.getPosition(objektId);

    // Verifiera att What3words används (medel prioritet)
    expect(position?.typ).toBe('What3words');
    expect(position?.precision).toBe('medel');
  });

  test('Should fallback to Adress if only Adress exists', async () => {
    // Rensa tidigare metadata
    await db.delete(metadata).where(eq(metadata.objektId, objektId));

    // Skapa endast Adress metadata
    await createMetadata({
      objektId,
      metadataTypNamn: 'Adress',
      varde: 'Test Street 123, Stockholm',
      arvsNedat: false,
    });

    // Hämta position
    const position = await geographicService.getPosition(objektId);

    // Verifiera att Adress används (lägst prioritet)
    expect(position?.typ).toBe('Adress');
    expect(position?.precision).toBe('grov');
  });
});
```

6.2 Integration Tests

Test Hela Flödet: `backend/src/__tests__/integration.test.ts`

```
import { describe, test, expect } from '@jest/globals';
import request from 'supertest';
import app from '../server'; // Export app från server.ts

describe('Integration Tests', () => {
  let objektId: number;
  let metadataId: number;

  test('POST /api/objects - Create object', async () => {
    const response = await request(app)
      .post('/api/objects')
      .send({
        namn: 'Integration Test Object',
        typ: 'fysisk',
        parentId: null,
      });

    expect(response.status).toBe(201);
    expect(response.body.namn).toBe('Integration Test Object');
    objektId = response.body.id;
  });

  test('GET /api/objects/:id - Get object', async () => {
    const response = await request(app)
      .get(`/api/objects/${objektId}`);

    expect(response.status).toBe(200);
    expect(response.body.id).toBe(objektId);
    expect(response.body.metadata).toBeInstanceOf(Array);
  });

  test('POST /api/objects/:id/metadata - Create metadata', async () => {
    const response = await request(app)
      .post(`/api/objects/${objektId}/metadata`)
      .send({
        metadataTypNamn: 'Antal',
        varde: 5,
        arvsNedat: false,
      });

    expect(response.status).toBe(201);
    expect(response.body.vardeInteger).toBe(5);
    metadataId = response.body.id;
  });

  test('GET /api/objects/:id/metadata - Get all metadata', async () => {
    const response = await request(app)
      .get(`/api/objects/${objektId}/metadata`);

    expect(response.status).toBe(200);
    expect(response.body.length).toBeGreaterThan(0);

    const antalMetadata = response.body.find((m: any) => m.typ.namn === 'Antal');
    expect(antalMetadata).toBeDefined();
    expect(antalMetadata.vardeInteger).toBe(5);
  });

  test('PUT /api/metadata/:id - Update metadata', async () => {
    const response = await request(app)
      .put(`/api/metadata/${metadataId}`)
      .send({
        varde: 10,
```

```
      });

    expect(response.status).toBe(200);
    expect(response.body.vardeInteger).toBe(10);
  });

  test('DELETE /api/metadata/:id - Delete metadata', async () => {
    const response = await request(app)
      .delete(`/api/metadata/${metadataId}`);

    expect(response.status).toBe(204);
  });

  test('DELETE /api/objects/:id - Delete object', async () => {
    const response = await request(app)
      .delete(`/api/objects/${objektId}`);

    expect(response.status).toBe(204);
  });
});
```

6.3 Test Cases från Kinab

Test Case 1: Willys Södermalm

```
describe('Willys Södermalm Test Case', () => {
  test('Should correctly handle Willys Södermalm structure', async () => {
    // 1. Hämta Willys Södermalm objekt
    const willysSodermalm = await getObjectWithAllMetadata(WILLYS_SODERMALM_ID);

    // 2. Verifiera objektdata
    expect(willysSodermalm.namn).toBe('Willys Södermalm');
    expect(willysSodermalm.typ).toBe('organisatorisk');

    // 3. Verifiera att kund-metadata finns
    const kundMetadata = willysSodermalm.metadata.find(m => m.typ.namn === 'Kund');
    expect(kundMetadata).toBeDefined();
    expect(kundMetadata?.source).toBe('local');

    // 4. Verifiera att adress finns
    const adressMetadata = willysSodermalm.metadata.find(m => m.typ.namn === 'Adress')
;
    expect(adressMetadata).toBeDefined();
    expect(adressMetadata?.vardeString).toBe('Götgatan 100, Stockholm');

    // 5. Hämta Pantrum (barn till Willys Södermalm)
    const pantrumId = 1001; // Från seed data
    const pantrum = await getObjectWithAllMetadata(pantrumId);

    // 6. Verifiera att Pantrum ärver adress från Willys Södermalm
    const pantrumAdress = pantrum.metadata.find(m => m.typ.namn === 'Adress');
    expect(pantrumAdress).toBeDefined();
    expect(pantrumAdress?.source).toBe('inherited');
    expect(pantrumAdress?.fromObject?.id).toBe(WILLYS_SODERMALM_ID);

    // 7. Verifiera att Pantrum har egen GPS
    const pantrumGPS = pantrum.metadata.find(m => m.typ.namn === 'GPS');
    expect(pantrumGPS).toBeDefined();
    expect(pantrumGPS?.source).toBe('local');

    // 8. Hämta geografisk position för Pantrum
    const position = await geographicService.getPosition(pantrumId);
    expect(position?.typ).toBe('GPS'); // GPS har högst prioritet
    expect(position?.varde).toBe('59.31563, 18.06493');
  });
});
```

Test Case 2: Stena Sessan (30 Soprum)

```
describe('Stena Sessan Test Case', () => {
  test('Should handle 30 soprum with shared and individual metadata', async () => {
    // Detta test simulerar Stena Sessan med 30 soprum
    // där varje soprum har gemensam metadata (adress, kund) men individuell metadata
(detaljtyp, antal)

    // 1. Skapa Stena Sessan (organisatorisk)
    const [stenaSessan] = await db.insert(clusterObjects).values({
      namn: 'Stena Sessan',
      typ: 'organisatorisk',
      parentId: null,
    }).returning();

    // 2. Lägg till gemensam metadata (ska ärvas)
    await createMetadata({
      objektId: stenaSessan.id,
      metadataTypNamn: 'Kund',
      varde: 'KUND_STENA',
      arvsNedat: true,
    });

    await createMetadata({
      objektId: stenaSessan.id,
      metadataTypNamn: 'Adress',
      varde: 'Stena Sessan, Norrköping',
      arvsNedat: true,
    });

    // 3. Skapa 30 soprum
    const soprumIds: number[] = [];
    for (let i = 1; i <= 30; i++) {
      const [soprum] = await db.insert(clusterObjects).values({
        namn: `Soprum ${i}`,
        typ: 'fysisk',
        parentId: stenaSessan.id,
      }).returning();

      soprumIds.push(soprum.id);

      // Lägg till individuell metadata
      await createMetadata({
        objektId: soprum.id,
        metadataTypNamn: 'Detaljtyp',
        varde: 'Soprum',
        arvsNedat: false,
      });

      await createMetadata({
        objektId: soprum.id,
        metadataTypNamn: 'Kod',
        varde: `${1000 + i}`,
        arvsNedat: false,
      });
    }

    // 4. Verifiera att alla soprum ärver gemensam metadata
    for (const soprumId of soprumIds) {
      const soprum = await getObjectWithAllMetadata(soprumId);

      // Kund ska vara ärvd
      const kundMetadata = soprum.metadata.find(m => m.typ.namn === 'Kund');
      expect(kundMetadata).toBeDefined();
```

```
      expect(kundMetadata?.source).toBe('inherited');
      expect(kundMetadata?.fromObject?.id).toBe(stenaSessan.id);

      // Adress ska vara ärvd
      const adressMetadata = soprum.metadata.find(m => m.typ.namn === 'Adress');
      expect(adressMetadata).toBeDefined();
      expect(adressMetadata?.source).toBe('inherited');

      // Detaljtyp ska vara lokal
      const detaljtypMetadata = soprum.metadata.find(m => m.typ.namn === 'Detaljtyp');
      expect(detaljtypMetadata).toBeDefined();
      expect(detaljtypMetadata?.source).toBe('local');

      // Kod ska vara lokal
      const kodMetadata = soprum.metadata.find(m => m.typ.namn === 'Kod');
      expect(kodMetadata).toBeDefined();
      expect(kodMetadata?.source).toBe('local');
    }

    // 5. Verifiera prestanda: Hämta alla 30 soprum ska ta < 2 sekunder
    const startTime = Date.now();
    await Promise.all(soprumIds.map(id => getObjectWithAllMetadata(id)));
    const endTime = Date.now();
    const duration = endTime - startTime;

    expect(duration).toBeLessThan(2000); // < 2 sekunder
  });
});
```

Test Case 3: BRF Tornet 4

```
describe('BRF Tornet 4 Test Case', () => {
  test('Should handle multi-level inheritance correctly', async () => {
    // Från seed data: BRF Tornet 4 > Tongatan 22 > Miljörum

    // 1. Hämta BRF Tornet 4
    const brfTornet4 = await getObjectWithAllMetadata(BRF_TORNET_4_ID);

    // Verifiera att BRF har kund och adress
    const brfKund = brfTornet4.metadata.find(m => m.typ.namn === 'Kund');
    expect(brfKund).toBeDefined();

    const brfAdress = brfTornet4.metadata.find(m => m.typ.namn === 'Adress');
    expect(brfAdress).toBeDefined();
    expect(brfAdress?.vardeString).toBe('Tongatan 22, Uppsala');

    // 2. Hämta Tongatan 22 (barn till BRF Tornet 4)
    const tongatan22 = await getObjectWithAllMetadata(TONGATAN_22_ID);

    // Verifiera att Tongatan 22 ärver kund och adress
    const tongatan22Kund = tongatan22.metadata.find(m => m.typ.namn === 'Kund');
    expect(tongatan22Kund).toBeDefined();
    expect(tongatan22Kund?.source).toBe('inherited');

    const tongatan22Adress = tongatan22.metadata.find(m => m.typ.namn === 'Adress');
    expect(tongatan22Adress).toBeDefined();
    expect(tongatan22Adress?.source).toBe('inherited');

    // 3. Hämta Miljörum (barn till Tongatan 22)
    const miljorum = await getObjectWithAllMetadata(MILJORUM_ID);

    // Verifiera att Miljörum ärver från grandparent (BRF Tornet 4)
    const miljorumKund = miljorum.metadata.find(m => m.typ.namn === 'Kund');
    expect(miljorumKund).toBeDefined();
    expect(miljorumKund?.source).toBe('inherited');
    expect(miljorumKund?.fromObject?.id).toBe(BRF_TORNET_4_ID);

    // 4. Verifiera att Miljörum har egen detaljtyp
    const miljorumDetaljtyp = miljorum.metadata.find(m => m.typ.namn === 'Detaljtyp');
    expect(miljorumDetaljtyp).toBeDefined();
    expect(miljorumDetaljtyp?.source).toBe('local');
    expect(miljorumDetaljtyp?.vardeString).toBe('Miljörum');
  });
});
```

Del 7: Checklista

✅ Databas

- [ ] PostgreSQL installerad och körande
- [ ] Databas skapad (`kinab`)
- [ ] Schema definierat (`src/db/schema.ts`)
- [ ] Drizzle konfigurerad (`drizzle.config.ts`)
- [ ] Migrationer körda
- [ ] Seed data inlagt

- [ ] Index skapade för prestanda
- [ ] Foreign keys konfigurerade

✅ Backend - Core

- [ ] Express server uppsatt
- [ ] CORS konfigurerat
- [ ] Error handling implementerat
- [ ] Environment variables konfigurerade
- [ ] Database connection pool uppsatt

✅ Backend - Queries

- [ ] `getObjectWithAllMetadata` - Hämta objekt med all metadata (inkl. ärvd)
- [ ] `getMetadataValue` - Hämta specifikt metadata-värde
- [ ] `createMetadata` - Skapa metadata
- [ ] `updateMetadata` - Uppdatera metadata
- [ ] `deleteMetadata` - Radera metadata
- [ ] `getCrossFertilizedMetadata` - Hämta korsbefruktad metadata
- [ ] `getGeographicPosition` - Hämta geografisk position med prioritering
- [ ] `getClusterTree` - Hämta klusterträd
- [ ] `findObjectsWithMetadata` - Hitta objekt med specifik metadata

✅ Backend - Services

- [ ] MetadataService - CRUD, nedärvning, korsbefruktning
- [ ] ClusterService - Träd-operationer, flytta objekt
- [ ] GeographicService - Positionering, upplösningsordning

✅ Backend - API Routes

- [ ] `GET /api/objects/:id` - Hämta objekt
- [ ] `POST /api/objects` - Skapa objekt
- [ ] `PUT /api/objects/:id` - Uppdatera objekt
- [ ] `DELETE /api/objects/:id` - Radera objekt
- [ ] `GET /api/objects/:id/metadata` - Hämta all metadata
- [ ] `POST /api/objects/:id/metadata` - Skapa metadata
- [ ] `PUT /api/metadata/:id` - Uppdatera metadata
- [ ] `DELETE /api/metadata/:id` - Radera metadata
- [ ] `GET /api/objects/:id/tree` - Hämta klusterträd
- [ ] `POST /api/metadata/crossfertilize` - Koppla metadata till metadata
- [ ] `GET /api/objects/:id/position` - Hämta geografisk position

✅ Frontend - Core

- [ ] Next.js projekt uppsatt
- [ ] TypeScript konfigurerat
- [ ] API client implementerad (`src/lib/api.ts`)
- [ ] Environment variables konfigurerade
- [ ] Metadata Context för state management

✅ Frontend - Komponenter

- [ ] ClusterTreeView - Visa klusterträd
- [ ] MetadataList - Visa metadata (lokala + ärvda)
- [ ] MetadataEditor - Redigera/skapa metadata
- [ ] ObjectView - Huvudvy för objekt
- [ ] Styling/UX för alla komponenter

✅ Testning

- [ ] Unit tests för metadata-nedärvning
- [ ] Unit tests för korsbefruktning
- [ ] Unit tests för geografisk upplösningsordning
- [ ] Integration tests för hela API
- [ ] Test cases för Willys Södermalm
- [ ] Test cases för Stena Sessan (30 soprum)
- [ ] Test cases för BRF Tornet 4

✅ Prestanda

- [ ] Query-prestanda testad (< 200ms för objekt med metadata)
- [ ] Index optimerade
- [ ] Caching implementerad där lämpligt
- [ ] N+1 queries eliminerade

✅ Dokumentation

- [ ] API-dokumentation skriven
- [ ] Databasschema dokumenterat
- [ ] Komponentdokumentation
- [ ] Deployment guide
- [ ] Användardokumentation

✅ Säkerhet & Validering

- [ ] Input-validering för alla API-endpoints
- [ ] Datatyp-validering för metadata
- [ ] SQL injection-skydd (via Drizzle ORM)
- [ ] Error messages ger inte bort känslig information

✅ Production Ready

- [ ] Environment variables för production
- [ ] Database backups konfigurerade
- [ ] Logging implementerat
- [ ] Monitoring uppsatt
- [ ] Deployment till production

Sammanfattning

Denna prompt ger Replit 3.0 allt som behövs för att implementera Mats vision om objektdata och metadata-systemet:

Vad som ska byggas:
- Minimalistiska objekt med endast 4 attribut (ID, Namn, Typ, Förälder)
- Flexibelt metadata-system med EAV-modell
- Nedärvningssystem för metadata
- Korsbefruktning (metadata kopplad till metadata)
- Geografisk upplösningsordning
- Komplett API och frontend

Teknologier:
- TypeScript
- PostgreSQL
- Drizzle ORM
- Express
- React/Next.js

Konkreta exempel:
- Willys Södermalm med Pantrum
- Stena Sessan med 30 soprum
- BRF Tornet 4 med multi-level nedärvning

Framgångskriterier:
- Funktionella kriterier uppfyllda
- Prestanda-mål uppnådda
- Alla tester gröna
- Koden är maintainable och skalbar

VIKTIGT: Denna prompt ska användas exakt som den är. All information som behövs för implementation finns här. Följ stegen i ordning och använd de fullständiga kodexemplen.

Lycka till med implementationen! 🚀